

Discrete Optimization

Optimal paths in bi-attribute networks with
fractional cost functions

H.M. Soroush *

Department of Statistics and Operations Research, College of Science, Kuwait University, P.O. Box 5969, Safat 13060, Kuwait

Received 25 June 2003; accepted 6 June 2007

Available online 19 September 2007

Abstract

An important routing problem is to determine an optimal path through a multi-attribute network which minimizes a cost function of path attributes. In this paper, we study an optimal path problem in a bi-attribute network where the cost function for path evaluation is fractional. The problem can be equivalently formulated as the “bi-attribute rational path problem” which is known to be NP-complete. We develop an exact approach to find an *optimal simple path* through the network when arc attributes are non-negative. The approach uses some path preference structures and elimination techniques to discard, from further consideration, those (partial) paths that cannot be parts of an optimal path. Our extensive computational results demonstrate that the proposed method can find optimal paths for large networks in very attractive times.

© 2007 Elsevier B.V. All rights reserved.

Keywords: Routing; Optimal path; Networks; Fractional/rational functions

1. Introduction

Many real-world decision making problems can be represented by network models featuring multiple and conflicting criteria (attributes). An important network problem is the multicriteria (multi-attributes) shortest path problem (MSPP) in which each arc possesses at least two criteria. MSPP has numerous applications in many diverse areas (e.g. [3,5,10,13–16,23,29,42]).

In the single criterion shortest path problem where each arc has only one attribute, efficient methods exist to find an optimal path between an origin–destination (O–D) pair of a network (e.g. [18]). However, due to the presence of conflicting criteria in MSPP, it is not generally likely to find a single optimal path, but rather a set of *Pareto-optimal*, *efficient*, or *non-dominated* paths must be determined. The Pareto-optimal paths are defined as those such that it is not possible to find another path with a better value in at least one criterion without worsening the value of at least one other criterion (e.g. [25]). Given the set of Pareto-optimal paths, one can

* Tel.: +965 498 5394; fax: +965 483 7332.

E-mail address: hsoroush@kuc01.kuniv.edu.kw

choose a path, not necessary the best for all criteria, but the most satisfactory. While the single criterion shortest path problem has a polynomial complexity bound (e.g. [18]), MSPP (including its bi-criteria case) is classified as NP-complete ([24]).

The available solution approaches for MSPP can be divided into four groups based on their prescribed strategies for identifying the Pareto-optimal paths. The first group includes the exact approaches (e.g. [4,8,9,26,31–34,39,43]). The second group consists of the approximate approaches (e.g. [27,44,45]). The third group comprises of utility (or cost) based approaches (e.g. [7,28,37,38]). The last group contains the interactive approaches (e.g. [11,12,17,25,40]).

In the bi-criteria case of MSPP, the objective can be to determine an optimal path which minimizes the ratio of the two criteria. For example, Ahuja [1] minimizes the ratio of cost and reliability where the reliability of a path is the product of the reliabilities of the arcs in that path. Fox [22] and Martins [30] minimize the ratio of cost and time. Martins [32] minimizes the ratio of cost and capacity. Meggido [35] suggests that the minimum ratio path problem, which belongs to a class of combinatorial optimization problems with rational objective functions, can be solved in polynomial time; however, Ahuja et al. [2] contradict Meggido's claim by showing that the minimum ratio path problem is NP-complete. Sherali et al. [42] present a discrete fractional programming formulation to determine a path through a bi-attribute network, where each arc has a probability of accident and its consequence (cost) in order to minimize the risk of low probability-high consequence accidents in transporting hazardous materials.

In this paper, we study an optimal path problem through a bi-attribute (bi-criteria) network with fractional cost function. The problem, which we refer to as OPFCF, is defined as follows. There is a network in which each arc has two non-negative attributes. The respective attributes of a path between an O–D pair of the network are the sum of the corresponding attributes of the arcs in that path. The cost function for path evaluation is fractional which is defined as the ratio of a linear function of one attribute and a linear function of the other attribute. The objective is to identify an *optimal simple path* between the O–D nodes that minimizes the fractional cost function over all paths. (A path is referred to as *simple* if it has no cycles or contains a node or an arc at most once which, for convenience, we call it a *path*.) We equivalently formulate OPFCF as the “bi-attribute rational path problem” which is known to be NP-complete (e.g. [2]), and show that, due the differences in the Pareto-optimality conditions for OPFCF and MSPP (including its bi-attribute case), the solution methods for MSPP cannot be applied to OPFCF. However, in this paper, we develop an exact approach that solves this challenging problem in a very reasonable amount of time.

There are many applications for OPFCF. For example, consider a stochastic communication network in which each arc has two attributes, say the mean and variance of the queueing delay of transmitting time through the arc. One may wish to find a path which minimizes the coefficient of variation of queueing delay (i.e., the ratio of standard deviation and mean of the delay). In a manufacturing environment, consider a production network in which each arc representing, for example, task or operation has two attributes such as input data and output data related to the task. Here, one may want to choose an optimal path (i.e., an optimal production strategy) whose productivity measure (i.e., the ratio of output data and input data) is maximized among all paths. In an investment problem, consider an investment network where each arc (investment) has two attributes; say the initial capital expenditure and the profit of an investment. Here, an investor may wish to choose an optimal path (i.e., an optimal investment strategy) that maximizes the rate of return (i.e., the ratio of profit and capital expended) over various investment opportunities.

The organization of this paper is as follows. In the following section, we define and formulate the optimal path problem in a bi-attribute network with fractional cost function. In Section 3, we introduce some path preference structures including the concepts of *permanent* and *temporary* preferences among paths. Section 4 addresses the concept of *relative preference ordering* and *temporary preference intervals* among paths and shows its importance in terms of eliminating paths and eventually identifying the candidates for an optimal path. Three algorithms (Algorithms 1–3) are developed in Section 5. Algorithms 1 and 2 are used to determine the *relative preference ordering* and *temporary preference intervals* for paths in some special networks. Algorithm 3 (an optimal path algorithm) utilizes the preference information obtained from Algorithms 1 and 2 in order to find an optimal path between the O–D nodes of a general bi-attribute network. Extensive computational results are presented in Section 6. Finally, Section 7 contains a summary and some concluding remarks.

2. Problem definition and formulation

Consider a network $G(V, A)$ where $V = \{v_1, \dots, v_n\}$ is its set of nodes and $A = \{a_1, \dots, a_m\}$ is its set of arcs. Suppose that nodes v_1 and v_n represent the origin–destination (O–D) nodes of G . Let $R^\ell = \{r_1, r_2, \dots, r_{|R^\ell|}\}$ be the set of all (*simple*) paths between the origin node v_1 and another node $v_\ell \in V$, $v_\ell \neq v_1$, where $|R^\ell|$ denotes the cardinality of R^ℓ . Associated with each arc $a_k \in A$ is a vector $[\xi_k, \psi_k]$ of two non-negative attributes (criteria) ξ_k and ψ_k . The attribute vector for a path $r_i \in R^\ell$ is defined as $[\Xi_i, \Psi_i] = \left[\sum_{a_k \in r_i} \xi_k, \sum_{a_k \in r_i} \psi_k \right]$. Each path $r_i \in R^\ell$ is evaluated using the fractional cost function $f(\xi, \psi) = (a\xi + b)/(c\psi + d)$ where $\xi, \psi \geq 0$; $a, c > 0$, and b and d are such that $a\xi + b \geq 0$ and $c\psi + d \geq 0$, i.e., $\min\{\xi_k, a_k \in A\} \geq -b/a$ for $b < 0$, and $\min\{\psi_k, a_k \in A\} \geq -d/c$ for $d < 0$. The objective is to find an optimal path that minimizes $f(\Xi_i, \Psi_i) = (a\Xi_i + b)/(c\Psi_i + d)$ over all paths $r_i \in R^n$ between the O–D nodes v_1 and v_n .

Note that the proposed optimal path problem in a bi-attribute network with fractional cost function (OPFCF) reduces to (i) the single criterion shortest path problem if $c = 0$ and $a, d > 0$ (e.g. [19]), and (ii) the single criterion longest path problem if $a = 0$ and $b, c > 0$ (e.g. [21]).

To evaluate paths in $G(V, A)$ using the cost function $f(\xi, \psi) = (a\xi + b)/(c\psi + d)$; $\xi, \psi \geq 0$; $a, c > 0$, we can equivalently utilize another network $\hat{G}(V, A)$ but using the cost function $f(x, y) = x/y$; $x, y \geq 0$. Network $\hat{G}$ has the same node set V , arc set A , and O–D nodes as those in G , but different arc attribute vectors $[x_k, y_k]$, $a_k \in A$, defined as

$$x_k = \begin{cases} a\xi_k + b, & \text{if arc } a_k \text{ is an outgoing arc from origin node } v_1, \\ a\xi_k, & \text{otherwise} \end{cases} \quad (1a)$$

and

$$y_k = \begin{cases} c\psi_k + d, & \text{if arc } a_k \text{ is an outgoing arc from origin node } v_1, \\ c\psi_k, & \text{otherwise.} \end{cases} \quad (1b)$$

The attribute vector $[X_i, Y_i]$ for each path $r_i \in R^\ell$ in $\hat{G}$ is defined as

$$[X_i, Y_i] = \left[\sum_{a_k \in r_i} x_k, \sum_{a_k \in r_i} y_k \right], \quad (2)$$

where x_k and y_k are given by (1). Employing $f(x, y) = x/y$, the cost $f(X_i, Y_i)$ for $r_i \in R^\ell$ in $\hat{G}$, using (1) and (2), is

$$f(X_i, Y_i) = X_i/Y_i = \sum_{a_k \in r_i} x_k / \sum_{a_k \in r_i} y_k = \left(a \sum_{a_k \in r_i} \xi_k + b \right) / \left(c \sum_{a_k \in r_i} \psi_k + d \right). \quad (3)$$

From (3) the cost $f(X_i, Y_i)$ for $r_i \in R^\ell$ in $\hat{G}$ is defined as the ratio of the attributes X_i and Y_i for r_i . This cost is also the same as that for $r_i \in R^\ell$ in G using the cost function $f(\xi, \psi) = (a\xi + b)/(c\psi + d)$. Therefore, OPFCF can be equivalently formulated as the “bi-attribute rational path problem” which is known to be NP-complete (e.g. [2]).

Given paths $r_i \neq r_j \in R^\ell$ from v_1 to v_ℓ of $\hat{G}(V, A)$, we say r_i is *preferred* to r_j (denoted by $r_i \succ r_j$) if $f(X_i, Y_i) \leq f(X_j, Y_j)$ or, using (3), if $X_i/Y_i \leq X_j/Y_j$. Then, a path $r^* \in R^n$ between the O–D nodes v_1 and v_n of $\hat{G}$ is called *optimal* if

$$r^* = \operatorname{argmin}_{r_i \in R^n} \{X_i/Y_i\} = \operatorname{argmin}_{r_i \in R^n} \left\{ \sum_{a_k \in r_i} x_k / \sum_{a_k \in r_i} y_k \right\}. \quad (4)$$

Let $\hat{r}$ and $\tilde{r}$ in R^n denote, respectively, the shortest path with respect to (w.r.t.) X -attribute and the longest path w.r.t. Y -attribute between the O–D nodes v_1 and v_n of $\hat{G}$, that is, let

$$\hat{r} = \operatorname{argmin}_{r_i \in R^n} \{X_i\} = \operatorname{argmin}_{r_i \in R^n} \left\{ \sum_{a_k \in r_i} x_k \right\},$$

and

$$\tilde{r} = \operatorname{argmax}_{r_i \in R^n} \{Y_i\} = \operatorname{argmax}_{r_i \in R^n} \left\{ \sum_{a_k \in r_i} y_k \right\}.$$

When $\hat{r} = \tilde{r}$, using (4), the path is also optimal for OPFCF (i.e., $r^* = \hat{r} = \tilde{r}$). However, in general, there is no such a path that simultaneously minimizes $\sum_{a_k \in r_i} x_k$ and maximizes $\sum_{a_k \in r_i} y_k$ over all $r_i \in R^n$.

We now have the following definitions.

Definition 1. Given paths $r_i \neq r_j \in R^\ell$ with attribute vectors $[X_i, Y_i]$ and $[X_j, Y_j]$, $r_i \succ r_j$ if either (i) $X_i < X_j$ and $Y_i \geq Y_j$ or (ii) $X_i \leq X_j$ and $Y_i > Y_j$.

Definition 2. A path $r_i \in R^\ell$ is said to be a *Pareto-optimal*, *efficient*, or *non-dominated* path in OPFCF if $r_i \succ r_j$ for any other path $r_j \in R^\ell$, that is, if either (i) $X_i < X_j$ and $Y_i \geq Y_j$ or (ii) $X_i \leq X_j$ and $Y_i > Y_j$ for any other path $r_j \in R^\ell$.

Given $r_i \neq r_j \in R^\ell$, if the Pareto-optimality conditions of Definition 2 hold, the non-Pareto or dominated path $r_j \in R^\ell$ can be eliminated; otherwise, both r_i and r_j are Pareto-optimal and must be kept for further consideration.

Similar to MSPP, we may consider solving OPFCF by generating all Pareto-optimal paths between the O–D nodes of $\hat{G}(V, A)$, and then choosing the Pareto path which minimizes the ratio X_i/Y_i as r^* . However, in general, identifying all Pareto-optimal paths may be computationally intractable since their number may grow exponentially with the number of nodes in the network (e.g. [25]) even though such paths may be found for some special networks with certain conditions imposed on their cycles [30]. In addition, since the Pareto-optimality conditions for OPFCF differ from those for MSPP (e.g. [43]), we cannot apply the solution procedures of MSPP to solve OPFCF.

In general, it is not also possible to develop a dynamic programming (DP) based approach (e.g. [46]) for solving OPFCF exactly. Before we show this via an example, let us define the concept of a *path segment addition*. Given a path $r_i \in R^\ell$ with attribute vector $[X_i, Y_i]$ from v_1 to an intermediate node v_ℓ , a path segment Δr_k with attribute vector $[\Delta X_k, \Delta Y_k]$ from v_ℓ to another node is referred to as a *path segment addition* to r_i resulting in an *augmented path* denoted by $r_{ik} = r_i \circ \Delta r_k$ with attribute vector $[X_{ik}, Y_{ik}] = [X_i + \Delta X_k, Y_i + \Delta Y_k]$.

Example 1. Consider the network $\hat{G}(V, A)$ of Fig. 1 where there are two parallel paths r_i and r_j between v_1 and v_ℓ (i.e., $R^\ell = \{r_i, r_j\}$) and a path segment addition Δr_k between v_ℓ and v_n . Suppose that the attribute vectors for r_i , r_j , and Δr_k are $[X_i, Y_i] = [3, 7]$, $[X_j, Y_j] = [5, 10]$, and $[\Delta X_k, \Delta Y_k] = [5, 3]$, respectively. Then, the attribute vectors for the two augmented paths $r_{ik} = r_i \circ \Delta r_k$ and $r_{jk} = r_j \circ \Delta r_k$ between v_1 and v_n are $[X_{ik}, Y_{ik}] = [8, 10]$ and $[X_{jk}, Y_{jk}] = [10, 13]$. Since $X_i/Y_i = 3/7 < X_j/Y_j = 1/2$, then $r_i \succ r_j$ at node v_ℓ ; however, $r_{jk} \succ r_{ik}$ at node v_n (since $X_{jk}/Y_{jk} = 10/13 < X_{ik}/Y_{ik} = 4/5$). That is, even though $r_i \succ r_j$ up to v_ℓ , this preference relation does not hold when r_i and r_j are augmented by Δr_k (i.e., $r_j \circ \Delta r_k \succ r_i \circ \Delta r_k$) at v_n . This is a clear violation of the principle of optimality of DP.

As demonstrated above, neither a DP-based approach nor an efficient Pareto-optimal path method can be developed to solve OPFCF exactly. Hence, a naive approach to identify an optimal path $r^* \in R^n$ is to (i)

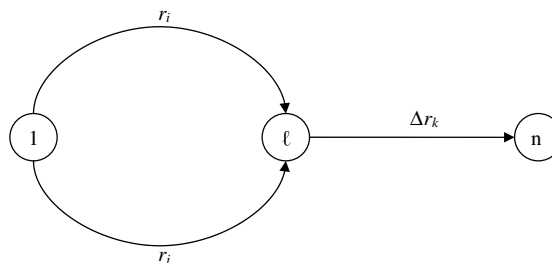


Fig. 1. A special bi-attribute network $\hat{G}(V, A)$ to illustrate various preference relations.

enumerate all $r_i \in R^n$, (ii) apply (1) and (2) to compute $[X_i, Y_i]$, $r_i \in R^n$, (iii) use (3) to calculate $f(X_i, Y_i)$, $r_i \in R^n$, and (iv) apply (4). However, since the problem $\min_{r_i \in R^n} \left\{ 1 / \sum_{a_k \in r_i} y_k \right\}$ is NP-complete (e.g. [2]), our problem $\min_{r_i \in R^n} \left\{ \sum_{a_k \in r_i} x_k / \sum_{a_k \in r_i} y_k \right\}$ is also NP-complete.

In what follows, we introduce a number of path preference structures and elimination techniques in order to determine and discard those (partial) paths that cannot be parts of the candidates for an optimal path between the O–D nodes of $\widehat{G}$. These structures and techniques can reduce the number of (partial) paths generated, evaluated, and stored when searching for an optimal path.

3. Path preferences in special bi-attribute networks

Consider again the network $\widehat{G}$ of Fig. 1, where the attribute vectors for r_i , r_j , and Δr_k are respectively $[X_i, Y_i]$, $[X_j, Y_j]$, and $[\Delta X_k, \Delta Y_k]$. Without loss of generality, let us subscript r_i and r_j such that $Y_i > Y_j$ and whenever $Y_i = Y_j$ such that $X_i \leq X_j$.

Based on Definition 1, we have the following property.

Property 1. *If $X_i \leq X_j$ and $Y_i \geq Y_j$, then $r_i \succ r_j$.*

Note that the conditions of Property 1 are only sufficient for $r_i \succ r_j$; thus, it is possible to have $r_i \succ r_j$ without these conditions being satisfied.

At node v_n of $\widehat{G}$, $r_{ik} \succ r_{jk}$ (i.e., $r_i \circ \Delta r_k \succ r_j \circ \Delta r_k$) if $X_{ik}/Y_{ik} \leq X_{jk}/Y_{jk}$ or

$$\Delta_{ij} + \Delta X_k(Y_j - Y_i) + \Delta Y_k(X_i - X_j) \leq 0; \quad (5)$$

else, $r_{jk} \succ r_{ik}$ where $\Delta_{ij} = X_i Y_j - X_j Y_i$. That is, using (5), $r_{ik} \succ r_{jk}$ if

$$\Delta X_k \geq \tau_{ij}(\Delta Y_k) = \frac{\Delta_{ij} + \Delta Y_k(X_i - X_j)}{Y_i - Y_j}; \quad (6)$$

else, $r_{jk} \succ r_{ik}$ where $Y_i > Y_j$ and $\tau_{ij}(\Delta Y_k)$, the threshold between r_i and r_j when augmented by Δr_k , is a function of ΔY_k . Note that when $Y_i = Y_j$, using (5), $r_{ik} \succ r_{jk}$ if $X_i \leq X_j$, and $r_{jk} \succ r_{ik}$ if $X_i > X_j$. Equivalently, when $Y_i = Y_j$, using (6), $\tau_{ij}(\Delta Y_k)$ goes to (i) $-\infty$ if $X_i < X_j$; thus $r_{ik} \succ r_{jk}$ (due to $\Delta X_k \geq 0$), and (ii) $+\infty$ if $X_i > X_j$; thus $r_{jk} \succ r_{ik}$.

The following additional properties can be derived from (6).

Property 2. *The threshold $\tau_{ij}(\Delta Y_k)$ is*

- (i) *a non-positive and non-increasing function of ΔY_k if $X_i \leq X_j$ and $Y_i \geq Y_j$;*
- (ii) *an increasing function of ΔY_k if $X_i > X_j$ and $Y_i \geq Y_j$.*

Proof. It immediately follows from (6). $\square$

Property 3. *If Property 2(i) holds, then (6) is always satisfied for any ΔX_k and ΔY_k , that is, $r_i \circ \Delta r_k \succ r_j \circ \Delta r_k$ for any Δr_k ; thus r_j can be eliminated from further consideration.*

Proof. Since $\Delta X_k \geq 0$ and $\tau_{ij}(\Delta Y_k)$ is a non-positive and non-increasing function of ΔY_k (due to Property 2(i)), then (6) always holds for any ΔX_k and ΔY_k . That is, $r_i \circ \Delta r_k \succ r_j \circ \Delta r_k$ for any Δr_k ; thus r_j can be discarded. $\square$

Definition 3. A path r_i is *permanently preferred* to a path r_j (denoted by $r_i \succ_p r_j$), $r_i \neq r_j \in R^\ell$, if $r_i \circ \Delta r_k \succ r_j \circ \Delta r_k$ for any Δr_k ; thus, r_j can be eliminated from further evaluation.

In general, since Property 3 does not hold, both r_i and r_j must be kept for further consideration. This can result in a large number of (partial) paths stored and evaluated in order to find $r^* \in R^n$. However, based on the following properties, we may be able to eliminate further (partial) paths that cannot become parts of the candidates for r^* .

Property 4. If $X_i > X_j$, $Y_i \geq Y_j$, and inequality (6) holds, then

- (i) $r_i \circ \Delta r_k \succ r_j \circ \Delta r_k$ as long as $\Delta X_k \geq \tau_{ij}(\Delta Y_k)$ or if $\Delta X_k \in [\tau_{ij}(\Delta Y_k), \infty)$, that is, $r_i \circ \Delta r_k \succ r_j \circ \Delta r_k$ for some Δr_k ; thus, neither r_i nor r_j can be discarded;
- (ii) $r_i \succ_p r_j$ and thus r_j can be discarded if $\Delta X_{\min}^\ell \geq \tau_{ij}(\Delta Y_{\max}^\ell)$ where $\Delta X_{\min}^\ell$ and $\Delta Y_{\max}^\ell$ denote, respectively, the length of the shortest path segment w.r.t. the X -attribute and the length of the longest path segment w.r.t. the Y -attribute from v_ℓ to v_n .

Proof. If Δr_k is augmented by another path segment $\Delta r'_k$ resulting in a longer segment $\Delta r_k \circ \Delta r'_k$ (with larger X and Y -attributes), then, based on Property 2(ii), the $\tau_{ij}(\cdot)$ for $\Delta r_k \circ \Delta r'_k$ becomes larger than that for Δr_k . Hence, the augmentation of Δr_k by $\Delta r'_k$ increases both left and right hand sides of inequality (6). This may reverse the direction of the inequality allowing $r_j \circ \Delta r_k \succ r_i \circ \Delta r_k$. Thus, $r_i \circ \Delta r_k \succ r_j \circ \Delta r_k$ if $\Delta X_k \geq \tau_{ij}(\Delta Y_k)$ or $\Delta X_k \in [\tau_{ij}(\Delta Y_k), \infty)$. That is, $r_i \circ \Delta r_k \succ r_j \circ \Delta r_k$ for some Δr_k , and neither r_i nor r_j can be eliminated. Moreover, based on Property 2(ii), $\tau_{ij}(\Delta Y_k)$ has the largest value when $\Delta Y_k = \Delta Y_{\max}^\ell$. Hence, if $\Delta X_k \geq \Delta X_{\min}^\ell \geq \tau_{ij}(\Delta Y_{\max}^\ell) \geq \tau_{ij}(\Delta Y_k)$, then $r_i \circ \Delta r_k \succ r_j \circ \Delta r_k$ for any Δr_k (i.e., $r_i \succ_p r_j$); thus r_j can be discarded. $\square$

Note that $\Delta X_{\min}^\ell$ and $\Delta Y_{\max}^\ell$ can be, respectively, computed by a single criterion shortest path algorithm (e.g. [19]) and either a CPM method if $\hat{G}$ is acyclic (e.g. [20]) or a single criterion longest path algorithm (e.g. [21]).

We now have the following additional definitions.

Definition 4. A path r_i is temporarily preferred to a path r_j (denoted by $r_i \succ_{\text{tp}} r_j$), $r_i \neq r_j \in R^\ell$, if $r_i \circ \Delta r_k \succ r_j \circ \Delta r_k$ for some Δr_k ; thus, neither r_i nor r_j can be discarded.

Definition 5. When $r_i \succ_{\text{tp}} r_j$; $r_i \neq r_j \in R^\ell$, the temporary preference interval (TPI) for path r_i when augmented by Δr_k (denoted by $\text{TPI}^i(\Delta Y_k) = [\tau_{ij}(\Delta Y_k), \infty)$) is the interval within which if ΔX_k falls then $r_i \circ \Delta r_k \succ r_j \circ \Delta r_k$; whereas $\text{TPI}^j(\Delta Y_k) = [0, \tau_{ij}(\Delta Y_k))$ is the TPI for path r_j .

Property 5. If $X_i > X_j$, $Y_i \geq Y_j$, and inequality (6) does not hold, then

- (i) $r_j \succ_{\text{tp}} r_i$ where $\text{TPI}^j(\Delta Y_k) = [0, \tau_{ij}(\Delta Y_k))$;
- (ii) $r_j \succ_p r_i$ and thus r_i can be eliminated if $\Delta X_{\max}^\ell < \tau_{ij}(\Delta Y_{\min}^\ell)$ where $\Delta X_{\max}^\ell$ and $\Delta Y_{\min}^\ell$ denote, respectively, the length of the longest path segment w.r.t. the X -attribute and the length of the shortest path segment w.r.t. the Y -attribute from v_ℓ to v_n .

Proof. Since (6) does not hold, that is, $\Delta X_k < \tau_{ij}(\Delta Y_k)$ which implies $r_j \circ \Delta r_k \succ r_i \circ \Delta r_k$, and $\tau_{ij}(\Delta Y_k)$ increases with ΔY_k (due to Property 2(ii)), any augmentation of Δr_k increases both ΔX_k and $\tau_{ij}(\Delta Y_k)$. This can reverse the direction of $\Delta X_k < \tau_{ij}(\Delta Y_k)$ allowing $r_i \circ \Delta r_k \succ r_j \circ \Delta r_k$. Thus, $r_j \circ \Delta r_k \succ r_i \circ \Delta r_k$ if ΔX_k falls in $[0, \tau_{ij}(\Delta Y_k))$ or if $\Delta X_k \in \text{TPI}^j(\Delta Y_k) = [0, \tau_{ij}(\Delta Y_k))$. That is, $r_j \circ \Delta r_k \succ r_i \circ \Delta r_k$ for some Δr_k , i.e., $r_j \succ_{\text{tp}} r_i$ with $\text{TPI}^j(\Delta Y_k) = [0, \tau_{ij}(\Delta Y_k))$. Furthermore, since $\tau_{ij}(\Delta Y_k)$ has the smallest value when $\Delta Y_k = \Delta Y_{\min}^\ell$ (due to Property 2(ii)), and (6) does not hold, then $r_j \circ \Delta r_k \succ r_i \circ \Delta r_k$. Hence, if $\Delta X_k \leq \Delta X_{\max}^\ell < \tau_{ij}(\Delta Y_{\min}^\ell) \leq \tau_{ij}(\Delta Y_k)$, $r_j \circ \Delta r_k \succ r_i \circ \Delta r_k$ for any Δr_k , i.e., $r_j \succ_p r_i$ and thus r_i can be discarded if $\Delta X_{\max}^\ell < \tau_{ij}(\Delta Y_{\min}^\ell)$. $\square$

Property 6. If Property 5(i) holds and $\Delta X_{\max}^\ell \in \text{TPI}^j(\Delta Y_k) = [0, \tau_{ij}(\Delta Y_k))$, then $r_j \succ_p r_i$ and r_i can be eliminated.

Proof. Using Property 5(i), if the location of destination node v_n is such that there does not exist any Δr_k whose ΔX_k violates $\Delta X_k \leq \Delta X_{\max}^\ell < \tau_{ij}(\Delta Y_k)$ (i.e., $\Delta X_k \in \text{TPI}^j(\Delta Y_k) = [0, \tau_{ij}(\Delta Y_k))$), then $r_j \circ \Delta r_k \succ r_i \circ \Delta r_k$ for any Δr_k , that is, $r_j \succ_p r_i$; thus r_i can be discarded. $\square$

The above path preference structure is a generalization of that of Mirchandani and Soroush [36] for an “optimal path problem in a probabilistic network”. However, in OPFCF, the conditions to establish path preferences and to determine temporary preference intervals are more complex. More importantly, the threshold $\tau_{ij}(\Delta Y_k)$ depends on Δr_k , whereas that of Mirchandani and Soroush is independent of Δr_k .

The following example illustrates the applications of above preference structures and elimination processes.

Example 2. Consider again the network $\hat{G}(V, A)$ of Fig. 1 but with two different sets of attribute vectors for r_i , r_j , and Δr_k . Let the first set of vectors be $[X_i, Y_i] = [7, 23]$, $[X_j, Y_j] = [11, 17]$, and $[\Delta X_k, \Delta Y_k]$. Since $X_i < X_j$ and $Y_i > Y_j$, then $r_i \succ_p r_j$ (due to Property 3); thus r_j can be eliminated. Now, consider the second set of vectors $[X_i, Y_i] = [7, 23]$, $[X_j, Y_j] = [1, 17]$, and $[\Delta X_k, \Delta Y_k] = [\Delta X_k, 3]$. Since $X_i > X_j$ and $Y_i > Y_j$, then there exists temporarily preferences between r_i and r_j (see Properties 4(i) and 5(i)). Subsequently, we have one of the following scenarios:

- (i) If $\Delta X_k \in \text{TPI}^i(\Delta Y_k) = [0, \tau_{ij}(\Delta Y_k)]$, where using (6), $\tau_{ij}(\Delta Y_k) = 19.0$, then $r_j \succ_p r_i$ (due to Property 5(i)); thus neither r_i nor r_j can be eliminated. However, if v_n is such that either $\Delta X_{\max}^\ell < \tau_{ij}(\Delta Y_{\min}^\ell)$ (see Property 5(ii)) or $\Delta X_{\max}^\ell \in \text{TPI}^j(\Delta Y_k) = [0, 19]$ (see Property 6), then $r_j \succ_p r_i$; thus r_i can be discarded.
- (ii) If $\Delta X_k \in \text{TPI}^i(\Delta Y_k) = [19, \infty)$, then $r_i \succ_p r_j$, and both r_i and r_j must be kept for further consideration (see Property 4(i)). However, if v_n is such that $\Delta X_{\min}^\ell \geq \tau_{ij}(\Delta Y_{\max}^\ell)$, then $r_i \succ_p r_j$ (see Property 4(ii)); thus r_j can be eliminated.

In general, since preferences among paths are not always permanent, an efficient approach cannot be developed to solve OPFCF exactly. In the section to follow, we introduce the concept of *relative preference ordering*. This is of significant importance in our development of an exact algorithm for OPFCF due to its role in eliminating additional (partial) paths; thus resulting in the evaluation of a smaller number of candidates for r^* .

4. Relative preference ordering

To discuss the concept of *relative preference ordering (RPO)*, consider the network $\hat{G}(V, A)$ of Fig. 2 where there are m parallel paths r_i , $i = 1, \dots, m$, between v_1 and v_ℓ (i.e., $R^\ell = \{r_1, \dots, r_m\}$) and n parallel path segments Δr_k , $k = 1, \dots, n$, between v_ℓ and v_n . The RPO allows us to evaluate only n instead of mn candidates at v_n in order to find r^* .

Without loss of generality, assume that the m paths in R^ℓ are subscripted such that $Y_1 \geq Y_2 \geq \dots \geq Y_m$. Suppose that the preferences among these paths are all temporary, that is, $r_i \circ \Delta r_k \succ r_j \circ \Delta r_k$ for some Δr_k , $k \in \{1, \dots, n\}$, $i < j = 1, \dots, m$.

To find the candidates for r^* , we compute the $\tau_{ij}(\Delta Y_k)$ for every pair of $r_i \neq r_j \in R^\ell$, $i < j$, when augmented by some Δr_k , $k \in \{1, \dots, n\}$, where

$$\tau_{ij}(\Delta Y_k) = \frac{\Delta_{ij} + \Delta Y_k(X_i - X_j)}{Y_i - Y_j}, \quad (7)$$

and $\Delta_{ij} = X_i Y_j - X_j Y_i$. This results in a total of $m(m-1)/2$ thresholds $\tau_{ij}(\Delta Y_k)$ for each Δr_k , $k \in \{1, \dots, n\}$. Arranging the thresholds for each Δr_k in an ascending order and comparing them with the corresponding ΔX_k , using (6), we can find an ordering among r_i , $i = 1, \dots, m$, such that the preferences among these paths change relative to ΔX_k . However, from the set of all arrangements among $\tau_{ij}(\Delta Y_k)$ for each Δr_k , only some will result in valid preference orderings among r_i , $i = 1, \dots, m$. For example, if there are three paths r_1 , r_2 , and r_3 to v_ℓ and only one path segment Δr_k between v_ℓ and v_n , we have six distinct arrangements among

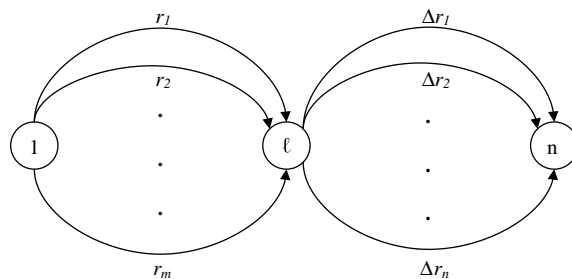


Fig. 2. A special bi-attribute network $\hat{G}(V, A)$ with m paths between nodes v_1 and v_ℓ and n path segments between nodes v_ℓ and v_n .

the three $\tau_{ij}(\Delta Y_k)$, $i < j = 1, 2, 3$. However, only the following two arrangements result in legitimate preference orderings among r_1 , r_2 , and r_3 .

Arrangement 1: $\tau_{12}(\Delta Y_k) < \tau_{13}(\Delta Y_k) < \tau_{23}(\Delta Y_k)$.

Arrangement 2: $\tau_{23}(\Delta Y_k) < \tau_{13}(\Delta Y_k) < \tau_{12}(\Delta Y_k)$.

The above two arrangements can be visualized as shown in Fig. 3. In Arrangement 1, as ΔX_k increases, using (6), the preference shifts from r_3 to r_1 ; thus, r_2 can be eliminated. To show this, we note that depending on the value of ΔX_k we can have one of the four cases: (1) $\Delta X_k < \tau_{12}(\Delta Y_k) < \tau_{13}(\Delta Y_k) < \tau_{23}(\Delta Y_k)$, (2) $\tau_{12}(\Delta Y_k) < \Delta X_k < \tau_{13}(\Delta Y_k) < \tau_{23}(\Delta Y_k)$, (3) $\tau_{12}(\Delta Y_k) < \tau_{13}(\Delta Y_k) < \Delta X_k < \tau_{23}(\Delta Y_k)$, and (4) $\tau_{12}(\Delta Y_k) < \tau_{13}(\Delta Y_k) < \tau_{23}(\Delta Y_k) < \Delta X_k$. In case (1), $r_2 \circ \Delta r_k \succ r_1 \circ \Delta r_k$ (since $\Delta X_k < \tau_{12}(\Delta Y_k)$), $r_3 \circ \Delta r_k \succ r_1 \circ \Delta r_k$ (since $\Delta X_k < \tau_{13}(\Delta Y_k)$), and $r_3 \circ \Delta r_k \succ r_2 \circ \Delta r_k$ (since $\Delta X_k < \tau_{23}(\Delta Y_k)$). These result in $r_3 \circ \Delta r_k \succ r_2 \circ \Delta r_k \succ r_1 \circ \Delta r_k$ implying that $r_3 \circ \Delta r_k$ is preferred. In case (2), we end up with $r_3 \circ \Delta r_k \succ r_1 \circ \Delta r_k \succ r_2 \circ \Delta r_k$ again implying that $r_3 \circ \Delta r_k$ is preferred. Cases (3) and (4), respectively, yield $r_1 \circ \Delta r_k \succ r_3 \circ \Delta r_k \succ r_2 \circ \Delta r_k$ and $r_1 \circ \Delta r_k \succ r_2 \circ \Delta r_k \succ r_3 \circ \Delta r_k$ which both imply that $r_1 \circ \Delta r_k$ is preferred. In none of the four cases $r_2 \circ \Delta r_k$ is preferred. Therefore, in Arrangement 1, as ΔX_k increases the preference shifts from r_3 to r_1 ; thus, r_2 can be discarded while r_3 and r_1 are kept for further consideration. The TPI's for r_3 and r_1 are, respectively, $\text{TPI}^3(\Delta Y_k) = [0, \tau_{13}(\Delta Y_k))$ and $\text{TPI}^1(\Delta Y_k) = [\tau_{13}(\Delta Y_k), \infty)$.

On the other hand, for Arrangement 2 we can similarly show that the preference moves from r_3 to r_2 and then to r_1 ; thus, no paths can be discarded. Here, the TPI's for r_3 , r_2 , and r_1 are, respectively, $\text{TPI}^3(\Delta Y_k) = [0, \tau_{23}(\Delta Y_k))$, $\text{TPI}^2(\Delta Y_k) = [\tau_{23}(\Delta Y_k), \tau_{12}(\Delta Y_k))$, and $\text{TPI}^1(\Delta Y_k) = [\tau_{12}(\Delta Y_k), \infty)$. To show that any other arrangement among $\tau_{ij}(\Delta Y_k)$, $i < j = 1, 2, 3$, results in a contradiction for some Δr_k , consider, e.g. the arrangement $\tau_{13}(\Delta Y_k) < \tau_{12}(\Delta Y_k) < \tau_{23}(\Delta Y_k)$. Assume that $\tau_{13}(\Delta Y_k) < \Delta X_k < \tau_{12}(\Delta Y_k)$. Since $\Delta X_k > \tau_{13}(\Delta Y_k)$, then $r_1 \circ \Delta r_k \succ r_3 \circ \Delta r_k$. Also, since $\Delta X_k < \tau_{12}(\Delta Y_k)$, $r_2 \circ \Delta r_k \succ r_1 \circ \Delta r_k$, and since $\Delta X_k < \tau_{23}(\Delta Y_k)$, $r_3 \circ \Delta r_k \succ r_2 \circ \Delta r_k$. These preference relations imply $r_2 \circ \Delta r_k \succ r_1 \circ \Delta r_k \succ r_3 \circ \Delta r_k \succ r_2 \circ \Delta r_k$ which is a contradiction.

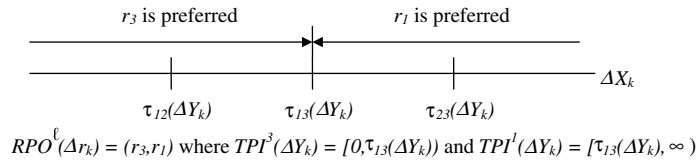
We now have the following property.

Property 7. Suppose that the thresholds $\tau_{ij}(\Delta Y_k)$ among temporarily preferred paths $r_i \neq r_j \in R^\ell = \{r_1, \dots, r_m\}$, $i < j$, for a path segment addition Δr_k , $k \in \{1, \dots, n\}$, when arranged in an ascending order for some r_i result in

$$\tau_{ij_m}(\Delta Y_k) < \tau_{ij_{m-1}}(\Delta Y_k) < \dots < \tau_{ij_2}(\Delta Y_k); \quad i < j_p \in \{1, \dots, m\}, \quad p = 2, \dots, m.$$

Then, eliminate path r_{j_p} from R^ℓ if at least one of the j_q indices among $\{j_q : \tau_{ij_p}(\Delta Y_k) < \tau_{ij_q}(\Delta Y_k)\}$ is greater than index j_p .

Arrangement 1:



Arrangement 2:

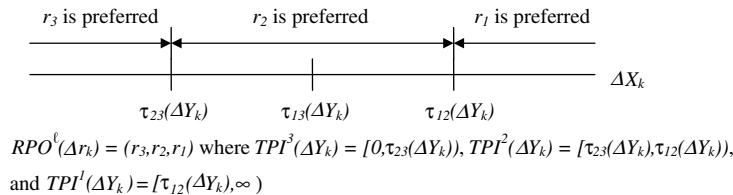


Fig. 3. RPO's and TPI's for paths r_1 , r_2 , and r_3 .

Proof. It is easily shown by utilizing arguments similar to those used in Arrangement 1 which resulted in the elimination of path r_2 . $\square$

To illustrate the application of **Property 7**, consider Arrangement 1 discussed above where the set of temporarily preferred paths is $R^\ell = \{r_1, r_2, r_3\}$. Since $\tau_{12}(\Delta Y_k) < \tau_{13}(\Delta Y_k)$ eliminate r_2 because the index 3 of r_3 in the order $\tau_{12}(\Delta Y_k) < \tau_{13}(\Delta Y_k)$ is greater than the index 2 of r_2 . This leaves behind r_3 and r_1 for further consideration.

To define the concept of *relative preference ordering*, let us assume that the preferences among paths in $R^\ell = \{r_1, \dots, r_m\}$ at node v_ℓ of network $\hat{G}$ of **Fig. 2** are all temporary. Suppose that the $\tau_{ij}(\Delta Y_k)$ between $r_i \neq r_j \in R^\ell$, $i < j$, when augmented by some Δr_k , $k \in \{1, \dots, n\}$, result in the order $\tau_{m-1,m}(\Delta Y_k) < \dots < \tau_{m-2,m-1}(\Delta Y_k) < \dots < \tau_{m-3,m-2}(\Delta Y_k) < \dots < \tau_{12}(\Delta Y_k)$.

Definition 6. The *relative preference ordering* among paths at v_ℓ for Δr_k (denoted by $\text{RPO}^\ell(\Delta r_k)$) is $\text{RPO}^\ell(\Delta r_k) = (r_m, \dots, r_1)$ indicating that as ΔX_k increases the preference shifts from r_m to r_{m-1} , $\dots$, and from r_2 to r_1 . The TPI's for paths in $\text{RPO}^\ell(\Delta r_k)$ are, respectively, $\text{TPI}^m(\Delta Y_k) = [0, \tau_{m-1,m}(\Delta Y_k)]$, $\text{TPI}^{m-1}(\Delta Y_k) = [\tau_{m-2,m-1}(\Delta Y_k), \tau_{m-3,m-2}(\Delta Y_k)]$, $\dots$, and $\text{TPI}^1(\Delta Y_k) = [\tau_{12}(\Delta Y_k), \infty)$.

According to **Definition 6**, for Arrangement 1 above, the *RDO* between r_1 and r_3 (r_2 is discarded due to **Property 7**), using $\tau_{13}(\Delta Y_k)$, is $\text{RPO}^\ell(\Delta r_k) = (r_3, r_1)$ where $\text{TPI}^3(\Delta Y_k) = [0, \tau_{13}(\Delta Y_k)]$ and $\text{TPI}^1(\Delta Y_k) = [\tau_{13}(\Delta Y_k), \infty)$, respectively. For Arrangement 2, the *RDO* among r_1 , r_2 , and r_3 , using the order $\tau_{23}(\Delta Y_k) < \tau_{12}(\Delta Y_k)$, is $\text{RPO}^\ell(\Delta r_k) = (r_3, r_2, r_1)$ where $\text{TPI}^3(\Delta Y_k) = [0, \tau_{23}(\Delta Y_k)]$, $\text{TPI}^2(\Delta Y_k) = [\tau_{23}(\Delta Y_k), \tau_{12}(\Delta Y_k)]$, and $\text{TPI}^1(\Delta Y_k) = [\tau_{12}(\Delta Y_k), \infty)$, respectively.

Given the $\text{RPO}^\ell(\Delta r_k)$ among the temporarily preferred paths in R^ℓ at node v_ℓ of the network $\hat{G}$ of **Fig. 2**, we may still be able to eliminate additional paths from $\text{RPO}^\ell(\Delta r_k)$ due to **Properties 4(ii)**, **5(ii)**, and **6**. For example, consider the $\text{RPO}^\ell(\Delta r_k) = (r_3, r_2, r_1)$ for Arrangement 2 discussed above. Suppose that the value for $\Delta X_{\max}^\ell$ is such that we end up with $\tau_{23}(\Delta Y) < \tau_{13}(\Delta Y) < \Delta X_{\max}^\ell < \tau_{12}(\Delta Y)$. Here, $r_2 \succ_p r_1$ (due to **Property 6**); thus r_1 can be eliminated producing $\text{RPO}^\ell(\Delta r_k) = (r_3, r_2)$. Moreover, utilizing the $\text{RPO}^\ell(\Delta r_k) = (r_3, r_2)$, we can have either

- (i) $r_2 \succ_p r_3$ if $\Delta X_{\min}^\ell \geq \tau_{23}(\Delta Y_{\max}^\ell)$ (due to **Property 4(ii)**); thus r_3 is discarded resulting in $\text{RPO}^\ell(\Delta r_k) = (r_2)$,
or
- (ii) $r_3 \succ_p r_2$ if $\Delta X_{\max}^\ell < \tau_{23}(\Delta Y_{\min}^\ell)$ (due to **Property 5(ii)**); thus r_2 is eliminated providing $\text{RPO}^\ell(\Delta r_k) = (r_3)$.

Next example illustrates our latest concepts.

Example 3. Consider the network $\hat{G}(V, A)$ of **Fig. 4** where the attribute vectors for r_1 , r_2 , r_3 , Δr_1 , and Δr_2 are given beside the paths. We show that at destination node v_n only two instead of six candidates are evaluated for an optimal path. Since $Y_1 > Y_2 > Y_3$ and $X_1 > X_2 > X_3$, preferences among r_1, r_2 , and r_3 are all temporary with $\text{RPO}^\ell(\Delta r_k) = (r_3, r_2, r_1)$, $k = 1, 2$. However, since $\Delta X_{\max}^\ell = 26 < \tau_{12}(\Delta Y_{\min}^\ell) = 43$ and $\Delta X_{\max}^\ell < \tau_{13}(\Delta Y_{\min}^\ell) = 33$, then $r_2 \succ_p r_1$ and $r_3 \succ_p r_1$, respectively (see **Property 5(ii)**); thus r_1 can be discarded. Hence, we end up with $\text{RPO}^\ell(\Delta r_k) = (r_3, r_2)$, $k = 1, 2$. For $\text{RPO}^\ell(\Delta r_1) = (r_3, r_2)$, we have $\text{TPI}^3(\Delta Y_1) = [0, \tau_{23}(\Delta Y_1)] = [0, 25]$ and $\text{TPI}^2(\Delta Y_1) = [\tau_{23}(\Delta Y_1), \infty) = [25, \infty)$. Since $\Delta X_1 = 26 \in \text{TPI}^2(\Delta Y_1)$, then $r_{21} = r_2 \circ \Delta r_1$ is a candidate to v_n . Similarly, for $\text{RPO}^\ell(\Delta r_2) = (r_3, r_2)$, we have $\text{TPI}^3(\Delta Y_2) = [0, \tau_{23}(\Delta Y_2)] = [0, 39]$ and $\text{TPI}^2(\Delta Y_2) =$

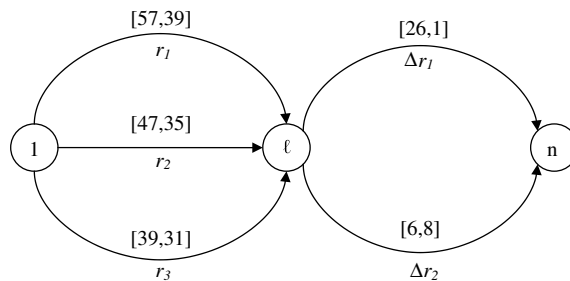


Fig. 4. A special bi-attribute network $\hat{G}(V, A)$ to illustrate RPO and TPI.

$[\tau_{23}(\Delta Y_2), \infty) = [39, \infty)$. Since $\Delta X_{\max}^\ell \in \text{TPI}^3(\Delta Y_2)$, then $r_3 \succ_p r_2$ (due to [Property 6](#)); thus r_2 is eliminated. This produces $\text{RPO}^\ell(\Delta r_2) = (r_3)$ which is used to get the other candidate $r_{32} = r_3 \circ \Delta r_2$ to v_n . Since $[X_{21}, Y_{21}] = [73, 36]$ for r_{21} and $[X_{32}, Y_{32}] = [45, 39]$ for r_{32} , then $r_{32} \succ r_{21}$ (due to [Property 1](#)). This leaves behind the only candidate $r_{32} = r_3 \circ \Delta r_2$ for r^* . Hence, $r^* = r_3 \circ \Delta r_2$ with cost $f(X_{32}, Y_{32}) = 45/39$.

As demonstrated above, RPO is very important in identifying an optimal path. It can result in a decrease in storage requirements since only the information on temporarily preferred paths in the RPO's are kept and used to compute the TPI's.

5. Optimal path algorithms

In this section, we first utilize the concepts developed so far to present an algorithm ([Algorithm 1](#)) in order to determine the RPO's and the TPI's for temporarily preferred paths in some special $\widehat{G}(V, A)$ networks. Based on [Algorithm 1](#), another algorithm ([Algorithm 2](#)) is then introduced to find the optimal paths between the O–D nodes of such networks. Later, we utilize [Algorithm 2](#) to propose an optimal path algorithm ([Algorithm 3](#)) for general networks.

5.1. An optimal path algorithm for special networks

Consider again the special network $\widehat{G}(V, A)$ of [Fig. 2](#) with the O–D nodes v_1 and v_n . Suppose that the preferences among paths in $R^\ell = \{r_1, \dots, r_m\}$ are all temporary, i.e., $r_i \succ_r r_j$, $i < j$; $r_i \neq r_j \in R^\ell$. Based on the path preference structures and elimination processes developed in [Sections 3 and 4](#), we below present an algorithm ([Algorithm 1](#)) which basically identifies and eliminates those (partial) paths to v_ℓ that can never be parts of any candidates for r^* at node v_n of $\widehat{G}(V, A)$. The algorithm then uses the RPO^ℓ and the TPI's for the temporarily preferred (partial) paths left behind at v_ℓ to generate the (partial) paths to the succeeding adjacent nodes of v_ℓ .

Algorithm 1

Begin

```

1. a.      For ( $v_\ell \in V$ , and  $v_\ell \neq v_1$ , and  $v_\ell \neq v_n$ ) do
              Compute  $\Delta X_{\min}^\ell, \Delta Y_{\min}^\ell, \Delta X_{\max}^\ell$ , and  $\Delta Y_{\max}^\ell$ ;
              End do
               $i := 1$ ;
               $R^\ell := \{r_1, \dots, r_m\}$ ;
b.      For ( $r_j \in R^\ell$ , and  $r_i \in R^\ell$ , and  $j > i$ ) do
              Calculate  $\Delta_{ij} = X_i Y_j - X_j Y_i$ ;
              Calculate  $\tau_{ij}(\Delta Y_{\min}^\ell) = [\Delta_{ij} + \Delta Y_{\min}^\ell (X_i - X_j)] / (Y_i - Y_j)$ ;
              Calculate  $\tau_{ij}(\Delta Y_{\max}^\ell) = [\Delta_{ij} + \Delta Y_{\max}^\ell (X_i - X_j)] / (Y_i - Y_j)$ ;
              End do
               $counter := 0$ ;
c.      If ( $\Delta X_{\min}^\ell \geq \tau_{ij}(\Delta Y_{\max}^\ell)$ ) then
               $R^\ell := R^\ell - \{r_j\}$ ;
               $counter := I$ ;
              End if
              If ( $\Delta X_{\max}^\ell < \tau_{ij}(\Delta Y_{\min}^\ell)$ ) then
               $R^\ell := R^\ell - \{r_i\} = \{r_1, \dots, r_p\}$ ;
               $counter := I$ ;
              End if
d.      If ( $counter = I$ ) then
               $i = j$ ;
              goto Step 1.b
              End if

```

- 2. a.** **For** ($k = 1, n$) **do**
 For ($r_j \in R^\ell$, and $r_i \in R^\ell$, and $j > i$) **do**
 Calculate $\tau_{ij}(\Delta Y_k) = \lceil \Delta_{ij} + \Delta Y_k(X_i - X_j) \rceil / (Y_i - Y_j)$;
 End do
 Order $\tau_{ij}(\Delta Y_k) \ni \tau_{ij_m}(\Delta Y_k) < \tau_{ij_{m-1}}(\Delta Y_k) < \dots < \tau_{ij_2}(\Delta Y_k)$; $i < j_p \in \{1, \dots, m\}$;
 $p = 2, \dots, m$;
b. **For** ($r_{j_p} \in R^\ell$, and $r_{j_q} \in R^\ell$, and $j_q > j_p$) **do**
 If ($\tau_{ij_p}(\Delta Y_k) < \tau_{ij_q}(\Delta Y_k)$) **then**
 $R^\ell := R^\ell - \{r_{j_p}\}$;
 counter := 1;
 End if
End do
 Order $R^\ell \ni \tau_{ij_m}(\Delta Y_k) < \tau_{ip_{m-1}}(\Delta Y_k) < \tau_{ip_{m-2}}(\Delta Y_k) < \dots$; where $j_m > p_{m-1} > p_{m-2} > \dots$;
3. a. **If** ($j_m = m$) **then**
 If ($\Delta Y_{\max}^\ell \geq \tau_{1m}(\Delta Y_k)$) **then**
 $RPO^\ell(\Delta r_k) := (r_m, r_1)$;
 If ($\Delta X_k \in TPI^m(\Delta Y_k) = (0, \tau_{1m}(\Delta Y_k))$) **then**
 $R^j := R^j \cup \{r_m \circ \Delta r_k\}$;
 Else
 $R^j := R^j \cup \{r_1 \circ \Delta r_k\}$;
 End if
 Else
 $RPO^\ell(\Delta r_k) := r_m$;
 $R^j := R^j \cup \{r_m \circ \Delta r_k\}$;
 End if
b. **Else**
 For ($t = m - 1, 1, -1$) **do**
 $i := p_t$;
 If (counter = 1) **goto** Step 1.b
 End do
 Order $\tau_{ij}(\Delta Y_k) \ni \tau_{q_t m}(\Delta Y_k) < \tau_{q_k q_t}(\Delta Y_k) < \dots < \tau_{q_t q_j}(\Delta Y_k) < \dots <$
 $\tau_{q_1 q_2}(\Delta Y_k) < \tau_{1 q_1}(\Delta Y_k)$;
 $RPO^\ell(\Delta r_k) := (r_m, r_{q_t}, r_{q_k}, \dots, r_{q_j}, r_{q_i}, \dots, r_{q_2}, r_{q_1}, r_1)$;
 End if
4. a. **If** ($\Delta X_{\max}^\ell \geq \tau_{1 q_1}(\Delta Y_k)$) **then**
 $RPO^\ell(\Delta r_k) := (r_m, r_{q_t}, r_{q_k}, \dots, r_{q_j}, r_{q_i}, \dots, r_{q_2}, r_{q_1}, r_1)$;
 If ($\Delta X_k \in TPI^m(\Delta Y_k) = (0, \tau_{q_t m}(\Delta Y_k))$) **then**
 $R^j := R^j \cup \{r_m \circ \Delta r_k\}$;
 Else if ($\Delta X_k \in TPI^{q_t}(\Delta Y_k) = (\tau_{q_t m}(\Delta Y_k), \tau_{q_k q_t}(\Delta Y_k))$) **then**
 $R^j := R^j \cup \{r_{q_t} \circ \Delta r_k\}$;
 $\vdots$
 Else if ($\Delta X_k \in TPI^{q_1}(\Delta Y_k) = (\tau_{q_1 q_2}(\Delta Y_k), \tau_{1 q_1}(\Delta Y_k))$) **then**
 $R^j := R^j \cup \{r_{q_1} \circ \Delta r_k\}$;
 Else
 $R^j := R^j \cup \{r_1 \circ \Delta r_k\}$;
 End if
b. **Else**
 $\Delta X_{\max}^\ell \ni$
 $\tau_{q_t m}(\Delta Y_k) < \tau_{q_k q_t}(\Delta Y_k) < \dots < \Delta X_{\max}^\ell < \tau_{q_t q_j}(\Delta Y_k) < \dots < \tau_{q_1 q_2}(\Delta Y_k) < \tau_{1 q_1}(\Delta Y_k) /$
 Eliminate paths $r_{q_i}, \dots, r_{q_2}, r_{q_1}, r_1$;
 $RPO^\ell(\Delta r_k) := (r_m, r_{q_t}, r_{q_k}, \dots, r_{q_{j+1}}, r_{q_j})$;

```

If ( $\Delta X_k \in \text{TPI}^m(\Delta Y_k) = (0, \tau_{q_\ell m}(\Delta Y_k))$ ) then
 $R^j := R^j \cup \{r_m \circ \Delta r_k\};$ 
Else if ( $\Delta X_k \in \text{TPI}^{q_\ell}(\Delta Y_k) = (\tau_{q_\ell m}(\Delta Y_k), \tau_{q_k q_\ell}(\Delta Y_k))$ ) then
 $R^j := R^j \cup \{r_{q_\ell} \circ \Delta r_k\};$ 
 $\vdots$ 
Else
 $R^j := R^j \cup \{r_{q_j} \circ \Delta r_k\};$ 
End if
End if
End do

```

Exit

Given the output of [Algorithm 1](#) for each Δr_k , $k = 1, \dots, n$, we determine which one of the $r_i \circ \Delta r_k$, $i \in \{1, \dots, m\}$, is a candidate for r^* to v_n of the special network of [Fig. 2](#). Since in this network there are n possible Δr_k , we then have n instead of mn candidates for r^* .

Step 1.a of [Algorithm 1](#) computes $\Delta X_{\min}^\ell$, $\Delta Y_{\min}^\ell$, $\Delta X_{\max}^\ell$, and $\Delta Y_{\max}^\ell$ which are, respectively, the lengths of the shortest path segments and the longest path segments w.r.t. X -attribute and Y -attribute from v_ℓ to v_n where $v_\ell \neq v_1$ and $v_\ell \neq v_n$. The step also sets the index $i := 1$ and $R^\ell := \{r_1, \dots, r_m\}$. Step 1.b utilizes (7) to compute the thresholds $\tau_{ij}(\Delta Y_{\min}^\ell)$ and $\tau_{ij}(\Delta Y_{\max}^\ell)$ between $r_i \in R^\ell$ and $r_j \in R^\ell$ where $i < j$. Step 1.c sets $R^\ell := R^\ell - \{r_j\}$ (i.e., eliminates r_j) if $\Delta X_{\min}^\ell \geq \tau_{ij}(\Delta Y_{\max}^\ell)$ (due to [Property 4\(ii\)](#)), and $R^\ell := R^\ell - \{r_i\}$ if $\Delta X_{\max}^\ell < \tau_{ij}(\Delta Y_{\min}^\ell)$ (due to [Property 5\(ii\)](#)). This can reduce the number of (partial) paths to be evaluated at v_ℓ resulting in $R^\ell = \{r_{j_1}, \dots, r_{j_p}\}$. Step 1.d checks to see whether any paths from $R^\ell = \{r_1, \dots, r_m\}$ have been discarded (i.e., *counter* = 1). If so, it sets $i := j_1$, and the algorithm returns to Step 1.b to see whether more paths can be eliminated from R^ℓ due to [Properties 4\(ii\)](#) and [5\(ii\)](#). Otherwise, the algorithm moves to Step 2. Step 2.a uses (7) to calculate $\tau_{ij}(\Delta Y_k)$ between $r_i \in R^\ell$ and $r_j \in R^\ell$, $i < j$ for each path segment addition Δr_k , $k \in \{1, \dots, n\}$, from v_ℓ to v_n , and then arranges these thresholds in an ascending order for r_i resulting in

$$\tau_{ij_m}(\Delta Y_k) < \tau_{ij_{m-1}}(\Delta Y_k) < \dots < \tau_{ij_2}(\Delta Y_k); \quad i < j_p \in \{1, \dots, m\}, \quad p = 2, \dots, m,$$

where $\tau_{st}(\Delta Y_k) > \tau_{ij_2}(\Delta Y_k)$; $i < s < t \in \{1, \dots, m\}$. Step 2.b uses [Property 7](#) to set $R^\ell := R^\ell - \{r_{j_p}\}$ if at least one of the j_q indices among $\{j_q : \tau_{ij_p}(\Delta Y_k) < \tau_{ij_q}(\Delta Y_k)\}$ is greater than index j_p . Let the indices of the remaining paths in R^ℓ be such that $\tau_{ij_m}(\Delta Y_k) < \tau_{ip_{m-1}}(\Delta Y_k) < \tau_{ip_{m-2}}(\Delta Y_k) < \dots < \tau_{ip_1}(\Delta Y_k)$ where $j_m > p_{m-1} > p_{m-2} > \dots > p_1$. When $j_m = m$, if $\Delta Y_{\max}^\ell \geq \tau_{1m}(\Delta Y_k)$, Step 3.a produces the $\text{RPO}^\ell(\Delta r_k) = (r_m, r_1)$, where $r_m \circ \Delta r_k$ is preferred for $\Delta X_k \in \text{TPI}^m(\Delta Y_k) = [0, \tau_{1m}(\Delta Y_k)]$ and $r_1 \circ \Delta r_k$ is preferred for $\Delta X_k \in \text{TPI}^1(\Delta Y_k) = [\tau_{1m}(\Delta Y_k), \infty)$; otherwise, the step yields the $\text{RPO}^\ell(\Delta r_k) = (r_m)$ where $r_m \circ \Delta r_k$ is preferred. When $j_m \neq m$, Step 3.b sets $i := p_t$ for each $t \in \{m-1, m-2, \dots, 1\}$ and repeats Step 1.b–Step 2.b until no more paths can be eliminated from R^ℓ . For each Δr_k , $k \in \{1, \dots, n\}$, suppose we end up with

$$\tau_{q_\ell m}(\Delta Y_k) < \tau_{q_k q_\ell}(\Delta Y_k) < \dots < \tau_{q_i q_j}(\Delta Y_k) < \dots < \tau_{q_1 q_2}(\Delta Y_k) < \tau_{1 q_1}(\Delta Y_k).$$

According to Step 4.a if $\Delta X_{\max}^\ell \geq \tau_{1 q_1}(\Delta Y_k)$, no paths can be discarded from R^ℓ yielding $\text{RPO}^\ell(\Delta r_k) = (r_m, r_{q_\ell}, r_{q_k}, \dots, r_{q_j}, r_{q_i}, \dots, r_{q_2}, r_{q_1}, r_1)$ where $r_m \circ \Delta r_k$ is preferred for $\Delta X_k \in \text{TPI}^m(\Delta Y_k) = [0, \tau_{q_\ell m}(\Delta Y_k)]$; $r_{q_\ell} \circ \Delta r_k$ is preferred for $\Delta X_k \in \text{TPI}^{q_\ell}(\Delta Y_k) = [\tau_{q_\ell m}(\Delta Y_k), \tau_{q_k q_\ell}(\Delta Y_k)]$; $\dots$; $r_{q_1} \circ \Delta r_k$ is preferred for $\Delta X_k \in \text{TPI}^{q_1}(\Delta Y_k) = [\tau_{q_1 q_2}(\Delta Y_k), \tau_{1 q_1}(\Delta Y_k)]$; and $r_1 \circ \Delta r_k$ is preferred for $\Delta X_k \in \text{TPI}^1(\Delta Y_k) = [\tau_{1 q_1}(\Delta Y_k), \infty)$. Otherwise, suppose $\Delta X_{\max}^\ell$ is such that we end up with:

$$\tau_{q_\ell m}(\Delta Y_k) < \tau_{q_k q_\ell}(\Delta Y_k) < \dots < \Delta X_{\max}^\ell < \tau_{q_i q_j}(\Delta Y_k) < \dots < \tau_{q_1 q_2}(\Delta Y_k) < \tau_{1 q_1}(\Delta Y_k).$$

Then, Step 4.b uses [Property 6](#) to eliminate $r_{q_i}, \dots, r_{q_2}, r_{q_1}, r_1$ from $\text{RPO}^\ell(\Delta r_k)$ producing $\text{RPO}^\ell(\Delta r_k) = (r_m, r_{q_\ell}, r_{q_k}, \dots, r_{q_{j+1}}, r_{q_j})$ where $r_m \circ \Delta r_k$ is preferred if $\Delta X_k \in \text{TPI}^m(\Delta Y_k) = [0, \tau_{q_\ell m}(\Delta Y_k)]$; $r_{q_\ell} \circ \Delta r_k$ is preferred if $\Delta X_k \in \text{TPI}^{q_\ell}(\Delta Y_k) = [\tau_{q_\ell m}(\Delta Y_k), \tau_{q_k q_\ell}(\Delta Y_k)]$; $\dots$; $r_{q_j} \circ \Delta r_k$ is preferred if $\Delta X_k \in \text{TPI}^{q_j}(\Delta Y_k) = [\tau_{q_j q_{j+1}}(\Delta Y_k), \infty)$. Consequently, [Algorithm 1](#) determines the n candidates for the optimal path to v_n .

We now present an optimal path algorithm ([Algorithm 2](#)) for the special network $G(V, A)$ of [Fig. 2](#). Basically, this algorithm first uses [Property 3](#) to identify and eliminate those (partial) paths that cannot be parts of

the candidates for r^* , and then invokes [Algorithm 1](#) to (i) discard additional (partial) paths, (ii) determine the candidates for r^* by using the RPO^ℓ and the TPI's of the remaining temporarily preferred paths at v_ℓ , and finally (iii) choose r^* from these candidates.

Algorithm 2

Begin

```

For ( $v_\ell \in V$  and  $v_\ell \neq v_1$ ) do
    counter := 0;
1. /indexing and arranging/
    a. Index the elements of  $R^\ell = \{r_1, \dots, r_m\} \ni Y_1 \geq Y_2 \geq \dots \geq Y_m$ ;
    b. Arrange  $X_i$  in non-increasing order  $\ni X_{i_1} \geq X_{i_2} \geq \dots \geq X_{i_m}$ ;
2. /Eliminating paths/
    Do  $k = 1, m - 1$ 
        Do  $p = k + 1, m$ 
            If ( $i_k > i_p$ ) then  $R^\ell := R^\ell - \{r_{i_k}\}$ ;
        End Do
    End Do
    If (counter = 1, and  $v_j = v_n$ ) goto Step 4
3. /Identifying the candidates paths/
    a. If ( $R^\ell = \{r_m\}$ ) then
        For ( $k = 1, n$ ) do
             $R^j := R^j \cup \{r_m \circ \Delta r_k\}$ ;
        End do
    End if
    b. If ( $|R^\ell| \geq 2$ ) then
        For ( $k = 1, n$ ) do
            Call Algorithm 1 ( $R^\ell, R^j$ )
        End do
    End if
    c. counter := 1;
    goto Step 1
4. /Finding the Optimal path/
     $r^* := \operatorname{argmin}_{r_i \in R^n} \{f(X_i, Y_i)\} = \operatorname{argmin}_{r_i \in R^n} \{X_i / Y_i\}$ ;
End do

```

Exit

Step 1 of [Algorithm 2](#) first subscripts $r_i \in R^\ell = \{r_1, \dots, r_m\}$ such that $Y_1 \geq Y_2 \geq \dots \geq Y_m$, and then arranges X_i in non-increasing order resulting in $X_{i_1} \geq X_{i_2} \geq \dots \geq X_{i_m}$. Utilizing [Property 3](#), Step 2 sets $R^\ell := R^\ell - \{r_{i_k}\}$ if the index i_k of $r_{i_k} \in R^\ell$ is larger than the index i_p of a $r_{i_p} \in R^\ell$ that appears later in the index order. If only r_m remains in R^ℓ , Step 3.a produces $R^j = \{r_m \circ \Delta r_k, k = 1, \dots, n\}$ as the set of candidates for r^* . Otherwise, Step 3.b invokes [Algorithm 1](#) to determine the $RPO^\ell(\Delta r_k)$, the respective $TPI'(\Delta Y_k)$, $i \in \{1, \dots, m\}$, for each Δr_k , and the set R^j of the n candidates $r_i \circ \Delta r_k$, $i \in \{1, \dots, m\}$, $k = 1, \dots, n$ to node v_j . Step 3.c returns to Step 1 followed by Step 2 to eliminate more (partial) paths from R^j due to [Properties 1 and 3](#). The algorithm now goes to Step 4 which uses (4) to find r^* .

Example 4. To illustrate [Algorithm 2](#), consider the network $\hat{G}(V, A)$ of [Fig. 5](#).

- Step 1.a subscripts paths in $R^\ell = \{r_1, r_2, r_3, r_4, r_5\}$ such that $Y_1 > Y_2 > Y_3 > Y_4 > Y_5$.
- Step 1.b yields $X_1 > X_3 > X_2 > X_4 > X_5$.
- Step 2 drops r_3 and sets $R^\ell = R^\ell - \{r_3\} = \{r_1, r_2, r_4, r_5\}$.
- Step 3.a invokes [Algorithm 1](#).

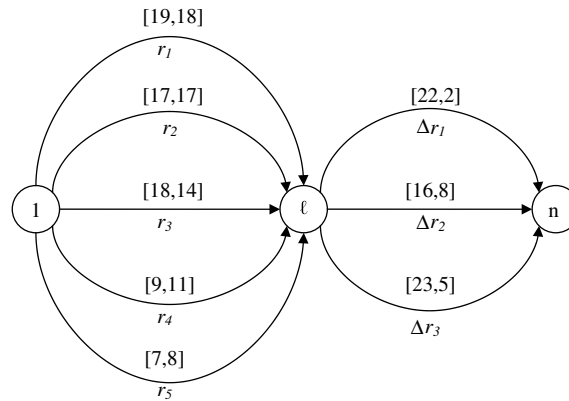


Fig. 5. A special bi-attribute network $\widehat{G}(V, A)$ to illustrate Algorithm 2.

- Step 1.a of Algorithm 1, calculates $\Delta X_{\min}^{\ell} = 16$, $\Delta X_{\max}^{\ell} = 23$, $\Delta Y_{\min}^{\ell} = 2$, and $\Delta Y_{\max}^{\ell} = 8$.
- Step 1.b computes $\tau_{12}(\Delta Y_{\min}^{\ell}) = 21$, $\tau_{12}(\Delta Y_{\max}^{\ell}) = 33$, $\tau_{14}(\Delta Y_{\min}^{\ell}) = 9.57$, $\tau_{14}(\Delta Y_{\max}^{\ell}) = 18.14$, $\tau_{15}(\Delta Y_{\min}^{\ell}) = 5$, and $\tau_{15}(\Delta Y_{\max}^{\ell}) = 12.2$.
- Since $\Delta X_{\min}^{\ell} > \tau_{15}(\Delta Y_{\max}^{\ell})$, Step 1.c discards r_5 and sets $R^{\ell} = R^{\ell} - \{r_5\} = \{r_1, r_2, r_4\}$.
- Step 2.a computes $\tau_{ij}(\Delta Y_k)$ for Δr_1 where $r_i \in R^{\ell}$ and $r_j \in R^{\ell}$, $i < j = 1, 2, 4$, resulting in $\tau_{12}(\Delta Y_1) = 21$, $\tau_{14}(\Delta Y_1) = 9.57$, and $\tau_{24}(\Delta Y_1) = 8.33$.
- Arranging $\tau_{ij}(\Delta Y_1)$, $i < j = 1, 2, 4$, in an ascending order, i.e., $\tau_{24}(\Delta Y_1) < \tau_{14}(\Delta Y_1) < \tau_{12}(\Delta Y_1)$, Step 3.b provides $\text{RPO}^{\ell}(\Delta r_1) = (r_4, r_2, r_1)$.
- Step 4.b uses $\text{RPO}^{\ell}(\Delta r_1)$ to obtain $\text{TPI}^4(\Delta Y_1) = [0, 8.33]$, $\text{TPI}^2(\Delta Y_1) = [8.33, 21]$, and $\text{TPI}^1(\Delta Y_1) = [21, \infty)$. Since $\Delta X_1 = 22 \in \text{TPI}^1(\Delta Y_1)$, $r_{11} = r_1 \circ \Delta r_1$ is the first candidate for r^* .
- Using $R^{\ell} = \{r_1, r_2, r_4\}$ and repeating Steps 2–4 of Algorithm 1 for Δr_2 , we get $\tau_{12}(\Delta Y_2) = 33$, $\tau_{14}(\Delta Y_2) = 18.14$, and $\tau_{24}(\Delta Y_2) = 16.33$.
- Arranging $\tau_{ij}(\Delta Y_2)$ for Δr_2 where $r_i \in R^{\ell}$ and $r_j \in R^{\ell}$, $i < j = 1, 2, 4$, in ascending order, i.e., $\tau_{24}(\Delta Y_2) < \tau_{14}(\Delta Y_2) < \tau_{12}(\Delta Y_2)$, Step 3 gives $\text{RPO}^{\ell}(\Delta r_2) = (r_4, r_2, r_1)$.
- Since $\Delta Y_{\max}^{\ell} < \tau_{12}(\Delta Y_2)$, Step 4.b eliminates r_1 , updates $\text{RPO}^{\ell}(\Delta r_2) = \text{RPO}^{\ell}(\Delta r_2) - \{r_1\} = (r_4, r_2)$, and determines $\text{TPI}^4(\Delta Y_2) = [0, 16.33]$ and $\text{TPI}^2(\Delta Y_2) = [16.33, \infty)$. Since $\Delta X_2 = 16 \in \text{TPI}^4(\Delta Y_2)$, $r_{42} = r_4 \circ \Delta r_2$ is the second candidate for r^* .
- Using $R^{\ell} = \{r_1, r_2, r_4\}$ and repeating Steps 2–4 of Algorithm 1 for Δr_3 , we obtain $\tau_{12}(\Delta Y_3) = 27$, $\tau_{14}(\Delta Y_3) = 13.85$, and $\tau_{24}(\Delta Y_3) = 12.34$.
- Step 3.b produces $\text{RPO}^{\ell}(\Delta r_3) = (r_4, r_2, r_1)$ from the ordering $\tau_{24}(\Delta Y_3) < \tau_{14}(\Delta Y_3) < \tau_{12}(\Delta Y_3)$.
- Since $\Delta X_{\max}^{\ell} < \tau_{12}(\Delta Y_3)$, Step 4.b discards r_1 , updates $\text{RPO}^{\ell}(\Delta r_3) = \text{RPO}^{\ell}(\Delta r_3) - \{r_1\} = (r_4, r_2)$, and finds $\text{TPI}^4(\Delta Y_3) = [0, 12.34]$ and $\text{TPI}^2(\Delta Y_3) = [12.34, \infty)$. Since $\Delta X_3 = 23 \in \text{TPI}^2(\Delta Y_3)$, $r_{23} = r_2 \circ \Delta r_3$ is the third candidate for r^* .
- Algorithm 1 terminates by providing the set of candidates $R^j = \{r_1 \circ \Delta r_1, r_4 \circ \Delta r_2, r_2 \circ \Delta r_3\}$ for r^* . Return to Algorithm 2.
- Given the attribute vectors $[41, 20]$, $[25, 19]$, and $[40, 22]$ for the three candidates $r_1 \circ \Delta r_1$, $r_4 \circ \Delta r_2$, and $r_2 \circ \Delta r_3$ in R^j , Step 3.c of Algorithm 2 repeats Steps 1 and 2. It discards $r_1 \circ \Delta r_1$, sets $R^n = R^j := R^j - \{r_1 \circ \Delta r_1\} = \{r_4 \circ \Delta r_2, r_2 \circ \Delta r_3\}$, and directs the algorithm to Step 4.
- Step 4 finds $r^* = r_4 \circ \Delta r_2 = \argmin_{r_i \in R^n} \{f(X_i, Y_i)\} = \argmin_{r_i \in R^n} \{X_i/Y_i\}$ as the optimal path between v_1 and v_n of the network of Fig. 5 with the minimum cost of 25/19.

5.2. An optimal path algorithm for general networks

Algorithm 2 is applicable only for a type of bi-attribute network as shown in Fig. 2. Before we present an optimal path algorithm (Algorithm 3) for general networks, let us introduce the concepts of *open*, *closed*, and

current nodes. In the algorithmic process, all nodes are initially labeled as *open* except the origin node v_1 which is labeled as *closed*. Once a node is closed, we never update its preference information (on RPO and TPI's), while for a *current* node we update its information until that node becomes closed. *Open* nodes are those for which no information has been obtained yet. The algorithm *closes* a current node only when none of the incoming arcs to it or its predecessors is from an open node. Consequently, the algorithm can be viewed as a label correcting one. Let the mutually exclusive sets of closed, current, and open nodes be denoted by B , C , and D , respectively, where $V = B \cup C \cup D$.

The input of Algorithm 3 includes the O–D nodes v_1 and v_n of network $\hat{G}(V, A)$ and the attribute vectors $[x_k, y_k]$, $a_k \in A$, while its output is the optimal path r^* between the O–D nodes. Basically, the algorithm invokes Algorithm 2 to compute and use the information on RPO and TPI's of the temporarily preferred (partial) paths to each node as well as the information on each arc between that node and every succeeding adjacent node in order to generate (partial) paths to that adjacent node. This process continues until all candidates to the destination node are found. Then, the candidate with the minimum cost is chosen as an optimal path. We now present our optimal path algorithm for a general network $\hat{G}(V, A)$.

Algorithm 3 (*An optimal path algorithm*).

Begin

1. $B := \{v_1\};$
 $C := \{\text{adjacent nodes to } v_1 \in B\};$
 $D := V - (B \cup C);$
 $R^1 = \emptyset;$
 $[X^1(r), Y^1(r)] = [0, 0], \forall r \in R^1;$
 2. a. **For** ($v_\ell \in B$, and $v_\ell \neq v_1$, and $v_j \in C$, and R^ℓ) **do**
 Call Algorithm 2(R^ℓ, R^j)
 Compute $[X^j(r_k^j), Y^j(r_k^j)], r_k^j \in R^j$ using Eq. (2);
 End do
 - b. **Repeat**
 Step 2.a for each $v_\ell (\neq v_n) \in C$ and each $v_j (\neq v_\ell) \in C$;
 Until (no more new (partial) paths can be included in $R^j, v_j \in C$);
 3. **If** $|C| \leq 2$ **then**
 goto Step 4
Else
 For ($v_\ell \in C$, and $v_j \in C$, and R^ℓ) **do**
 Call Algorithm 2(R^ℓ, R^j)
 Compute $[X^j(r_k^j), Y^j(r_k^j)], r_k^j \in R^j$ using Eq. (2);
 End do
End if
 4. Close each $v_\ell \in C$ if v_ℓ and none of its predecessors $v_j \in C$ have any incoming arcs from a $v_k \in D$;
If $D = \emptyset$ **then**
 $B := B \cup C;$
 goto Step 5
Else
 a. **If** (all nodes $v_\ell \in C$ can be closed) **then**
 $B := B \cup C, C := \{\text{all } v_k \in D \text{ adjacent to each } v_j \in B\};$
 b. **Else** (no node $v_\ell \in C$ can be closed)
 $C := C \cup \{\text{all } v_k \in D \text{ adjacent to each } v_j \in C\};$
 End if
 $D := V - (B \cup C);$
 goto Step 2
End if
 5. $r^* := \underset{r_k^n \in R^n}{\operatorname{argmin}} \{f(X^n(r_k^n), Y^n(r_k^n))\} = \underset{r_k^n \in R^n}{\operatorname{argmin}} \{X^n(r_k^n) / Y^n(r_k^n)\};$
- Exit**

Step 1 of **Algorithm 3** considers the origin node v_1 as *closed* (i.e., $B = \{v_1\}$), its adjacent nodes as *current* (i.e., $C = \{\text{adjacent nodes to } v_1 \in B\}$), and the remaining nodes as *open* (i.e., $D = V - (B \cup C)$). It also defines the set of paths to v_1 as empty (i.e., $R^1 = \emptyset$), and initializes the X -attribute and Y -attribute of the preferred (partial) path in R^1 as $[X^1(r), Y^1(r)] = [0, 0]$. Step 2.a applies **Algorithm 2** to the set R^ℓ of temporarily preferred (partial) paths at each $v_\ell \in B$ to construct the preferred augmented (partial) paths $r'_k, k = 1, \dots$, in R^j from $v_\ell \in B$ to each of its adjacent node $v_j \in C$ by augmenting the corresponding (partial) path in R^ℓ with the path segment addition (arc) (v_ℓ, v_j) . It then uses (2) to compute the attribute vectors $[X^j(r'_k), Y^j(r'_k)] = [\sum_{a_\ell \in r'_k} x_\ell, \sum_{a_\ell \in r'_k} y_\ell]$, $r'_k \in R^j$.

Step 2.b repeats step 2.a for each $v_\ell (\neq v_n) \in C$ and each $v_j (\neq v_\ell) \in C$ until no further new preferred (partial) paths to $v_j \in C$ can be generated and included in R^j . If the number of *current* nodes is at most two, Step 3 directs the algorithm to Step 4 because all possible path segment additions (arcs) have been added to paths in R^ℓ in order to find the preferred augmented (partial) paths $r'_k \in R^j$; otherwise, **Algorithm 2** is again applied to the (partial) paths in $R^\ell, v_\ell \in C$, in order to update $R^j, v_j \in C$, by determining whether the segments (to v_ℓ) of (partial) paths in $R^j, v_j \in C$, can be replaced by any other segments (to v_ℓ) now available due to Step 2.b. Here, Step 3 also uses (2) to compute $[X^j(r'_k), Y^j(r'_k)]$ for any new updated paths $r'_k \in R^j$. Step 4 closes each $v_\ell \in C$ if v_ℓ and none of its predecessors $v_j \in C$ have any incoming arcs from a $v_k \in D$ (i.e., if all possible (partial) paths $r'_k \in R^\ell$ (to v_ℓ) are identified and evaluated). If no *open* node exists (i.e., $D = \emptyset$), all nodes including v_n are *closed* (i.e., $B := B \cup C$), and the algorithm moves to Step 5 to find r^* from the set of candidates to v_n . Otherwise, if each $v_\ell \in C$ is such that neither itself nor one of its predecessors $v_j \in C$ have incoming arcs from a $v_k \in D$ (i.e., if all (partial) paths to $v_\ell \in C$ are found and evaluated), then Step 4.a *closes* $v_\ell \in C$ (i.e., $B := B \cup C$), considers the adjacent *open* nodes $v_k \in D$ to each $v_j \in B$ as *current* (i.e., $C := \{\text{all } v_k \in D \text{ adjacent to each } v_j \in B\}$), sets $D := V - (B \cup C)$, and returns the algorithm to Step 2. Else, if no $v_\ell \in C$ can be closed (i.e., if each $v_\ell \in C$ is such that either itself or one of its predecessors $v_j \in C$ have incoming arcs from a $v_k \in D$), then Step 4.b considers the adjacent *open* nodes $v_k \in D$ to each $v_\ell \in C$ as *current* (i.e., $C := C \cup \{\text{all } v_k \in D \text{ adjacent to each } v_j \in C\}$) where now $v_j \in C$ and its adjacent nodes are $v_i \in D$; sets $D := V - (B \cup C)$, and returns the algorithm to Step 2. Finally, Step 5 chooses the candidate at v_n with the minimum $f(X^n(r'_k), Y^n(r'_k)) = X^n(r'_k)/Y^n(r'_k)$, $r'_k \in R^n = \{r'_k, k = 1, \dots\}$ as the optimal path r^* between the O–D nodes v_1 and v_n ; and terminates the algorithm.

Algorithm 3 may be extended to identify the optimal paths from the origin node v_1 to all other nodes of the network. Here, for each destination node, we perform Steps 1–5 of the algorithm in parallel.

In the worst case **Algorithm 3** is exponential in the number of nodes in the network, since it is conceivable that for some networks no (partial) paths between the O–D nodes may be eliminated. However, due to our path elimination processes, from the establishment of permanently preferred preferences (see Steps 2–4 of **Algorithm 3**), a significant number of (partial) paths are discarded early in the algorithm. The following illustrative example and the computational results of Section 6 clearly demonstrate the significant importance of these elimination processes in discarding (partial) paths.

Next example illustrates the proposed optimal path algorithm (**Algorithm 3**).

Example 5. Consider the network $\widehat{G}(V, A)$ of Fig. 6 in which the arcs (v_2, v_3) , (v_2, v_4) , (v_3, v_4) , and (v_5, v_6) are undirected, the O–D nodes are v_1 and v_7 , and the attribute vector $[x_k, y_k]$ for each arc $a_k \in A$ is given beside that arc.

Algorithm 3 proceeds as follows.

- Step 1 sets $B = \{v_1\}$, $C = \{v_2, v_3, v_4\}$, $D = \{v_5, v_6, v_7\}$, $R^1 = \emptyset$, and $[X^1(r), Y^1(r)] = [0, 0]$.
- Since $v_1 \in B$, Step 2.a calls **Algorithm 2** to determine $R^\ell, v_\ell \in C = \{v_2, v_3, v_4\}$ resulting in

$$R^2 = \{v_1 - v_2\} \quad \text{with } \text{RPO}^2(\Delta r_k) = (v_1 - v_2),$$

$$R^3 = \{v_1 - v_3\} \quad \text{with } \text{RPO}^3(\Delta r_k) = (v_1 - v_3),$$

$$R^4 = \{v_1 - v_4\} \quad \text{with } \text{RPO}^4(\Delta r_k) = (v_1 - v_4),$$

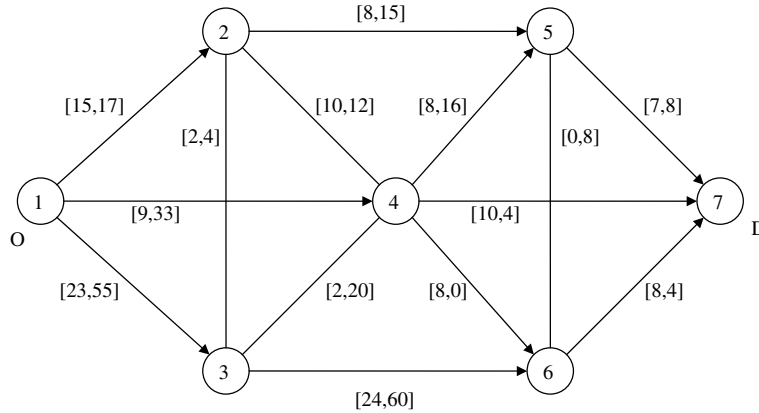


Fig. 6. A bi-attribute network $\hat{G}(V, A)$ to illustrate the optimal path algorithm (Algorithm 3).

where using (2),

$$[X^2(v_1 - v_2), Y^2(v_1 - v_2)] = [15, 17],$$

$$[X^3(v_1 - v_3), Y^3(v_1 - v_3)] = [23, 55],$$

$$[X^4(v_1 - v_4), Y^4(v_1 - v_4)] = [9, 33].$$

- Step 2.b repeats Step 2.a for each $v_\ell \in C = \{v_2, v_3, v_4\}$ and each $v_j (\neq v_\ell) \in C$ to obtain the temporarily preferred (partial) paths in R^j yielding

$$R^2 = \{v_1 - v_2, v_1 - v_3 - v_2, v_1 - v_4 - v_2\} \quad \text{with } \text{RPO}^2(\Delta r_k) = (v_1 - v_3 - v_2, v_1 - v_4 - v_2, v_1 - v_2),$$

$$R^3 = \{v_1 - v_3, v_1 - v_4 - v_3\} \quad \text{with } \text{RPO}^3(\Delta r_k) = (v_1 - v_3, v_1 - v_4 - v_3),$$

$$R^4 = \{v_1 - v_4, v_1 - v_3 - v_4\} \quad \text{with } \text{RPO}^4(\Delta r_k) = (v_1 - v_3 - v_4, v_1 - v_4)$$

where

$$[X^2(v_1 - v_3 - v_2), Y^2(v_1 - v_3 - v_2)] = [25, 59],$$

$$[X^2(v_1 - v_4 - v_2), Y^2(v_1 - v_4 - v_2)] = [19, 45],$$

$$[X^3(v_1 - v_4 - v_3), Y^3(v_1 - v_4 - v_3)] = [11, 53],$$

$$[X^4(v_1 - v_3 - v_4), Y^4(v_1 - v_3 - v_4)] = [25, 75].$$

- Since $|C| = 3 > 2$, Step 3 invokes Algorithm 2 and uses the temporarily preferred (partial) paths in R^ℓ , $v_\ell \in C = \{v_2, v_3, v_4\}$ to construct the preferred augmented (partial) paths in R^j at each of adjacent nodes $v_j (\neq v_\ell) \in C$ of $v_\ell \in C$. This yields

$$R^2 = \{v_1 - v_4 - v_3 - v_2\} \quad \text{with } \text{RPO}^2(\Delta r_k) = (v_1 - v_4 - v_3 - v_2),$$

$$R^3 = \{v_1 - v_4 - v_3\} \quad \text{with } \text{RPO}^3(\Delta r_k) = (v_1 - v_4 - v_3),$$

$$R^4 = \{v_1 - v_4, v_1 - v_3 - v_4\} \quad \text{with } \text{RPO}^4(\Delta r_k) = (v_1 - v_4, v_1 - v_3 - v_4),$$

where

$$[X^2(v_1 - v_4 - v_3 - v_2), Y^2(v_1 - v_4 - v_3 - v_2)] = [13, 57].$$

Table 1 shows the TPI's among paths in $\text{RPO}^4(\Delta r_k)$ for various path segment additions Δr_k .

- Step 4 closes all current nodes $v_\ell \in C = \{v_2, v_3, v_4\}$. Step 4.a sets $B := B \cup C = \{v_1, v_2, v_3, v_4\}$, $C := \{v_5, v_6, v_7\}$, $D := V - (B \cup C) = \emptyset$, and returns the algorithm to Step 2.
- Step 2.a now applies Algorithm 2 to the temporarily preferred (partial) paths in R^ℓ , $v_\ell (\neq v_1) \in B = \{v_1, v_2, v_3, v_4\}$, to determine the set R^j of preferred augmented (partial) paths to each of adjacent node $v_j \in C = \{v_5, v_6, v_7\}$ of v_ℓ . This results in

Table 1

RPO's and TPI's among the temporarily preferred (partial) paths at each node when finding the optimal path between the O–D nodes v_1 and v_7 of the bi-attribute network $\widehat{G}(V, A)$ of Fig. 6

Node v_ℓ	Δr_k : arc $v_\ell - v_j$	$\text{RPO}^\ell(\Delta r_k)$	$\text{TPI}(\Delta Y_k)$	ΔX_k	Candidate paths to v_j
v_2	$v_2 - v_3$	— ^a	—	2	—
	$v_2 - v_4$	— ^a	—	10	—
	$v_2 - v_5$	$v_1 - v_4 - v_3 - v_2$	—	8	$v_1 - v_4 - v_3 - v_2 - v_5$
v_3	$v_3 - v_2$	$v_1 - v_4 - v_3$	—	2	$v_1 - v_4 - v_3 - v_2$
	$v_3 - v_4$	— ^a	—	2	—
	$v_3 - v_6$	$v_1 - v_4 - v_3$	—	24	$v_1 - v_4 - v_3 - v_6$
v_4	$v_4 - v_2$	$v_1 - v_4, v_1 - v_3 - v_4$	$[0, 8.14), [8.14, \infty)$	10	$v_1 - v_3 - v_4 - v_2$
	$v_4 - v_3$	$v_1 - v_4, v_1 - v_3 - v_4$	$[0, 11.19), [11.19, \infty)$	2	$v_1 - v_4 - v_3$
	$v_4 - v_5$	$v_1 - v_4, v_1 - v_3 - v_4$	$[0, 9.66), [9.66, \infty)$	8	$v_1 - v_4 - v_5$
	$v_4 - v_6$	$v_1 - v_4, v_1 - v_3 - v_4$	$[0, 3.57), [3.57, \infty)$	8	$v_1 - v_3 - v_4 - v_6$
	$v_4 - v_7$	$v_1 - v_4, v_1 - v_3 - v_4$	$[0, 5.09), [5.09, \infty)$	10	$v_1 - v_3 - v_4 - v_7$
v_5	$v_5 - v_6$	$v_1 - v_4 - v_3 - v_2 - v_5$	—	0	$v_1 - v_4 - v_3 - v_2 - v_5 - v_6$
	$v_5 - v_7$	$v_1 - v_4 - v_3 - v_6 - v_5$	—	7	$v_1 - v_4 - v_3 - v_6 - v_5 - v_7$
v_6	$v_6 - v_5$	$v_1 - v_4 - v_3 - v_2 - v_5$	—	0	$v_1 - v_4 - v_3 - v_6 - v_5$
	$v_6 - v_7$	$v_1 - v_4 - v_3 - v_2 - v_5 - v_6$	—	8	$v_1 - v_4 - v_3 - v_2 - v_5 - v_6 - v_7$
v_7	—	$v_1 - v_4 - v_3 - v_6 - v_5 - v_7$	—	—	$v_1 - v_4 - v_3 - v_6 - v_5 - v_7$

^a The RPO did not contain any path.

$$R^5 = \{v_1 - v_4 - v_3 - v_2 - v_5\} \quad \text{with } \text{RPO}^5(\Delta r_k) = (v_1 - v_4 - v_3 - v_2 - v_5),$$

$$R^6 = \{v_1 - v_4 - v_3 - v_6\} \quad \text{with } \text{RPO}^6(\Delta r_k) = (v_1 - v_4 - v_3 - v_6),$$

$$R^7 = \{v_1 - v_3 - v_4 - v_7\} \quad \text{with } \text{RPO}^7(\Delta r_k) = (v_1 - v_3 - v_4 - v_7),$$

where

$$[X^5(v_1 - v_4 - v_3 - v_2 - v_5), Y^5(v_1 - v_4 - v_3 - v_2 - v_5)] = [21, 72],$$

$$[X^6(v_1 - v_4 - v_3 - v_6), Y^6(v_1 - v_4 - v_3 - v_6)] = [35, 113],$$

$$[X^7(v_1 - v_3 - v_4 - v_7), Y^7(v_1 - v_3 - v_4 - v_7)] = [35, 79].$$

Table 1 displays the $\text{RPO}^\ell(\Delta r_k)$, $v_\ell \in C = \{v_5, v_6, v_7\}$, and the TPI's among paths in $\text{RPO}^\ell(\Delta r_k)$ for various path segment additions Δr_k .

- Step 2.b repeats Step 2.a for each $v_\ell (\neq v_n) \in C = \{v_5, v_6, v_7\}$ and each $v_j (\neq v_\ell) \in C$ and finds

$$R^5 = \{v_1 - v_4 - v_3 - v_2 - v_5, v_1 - v_4 - v_3 - v_6 - v_5\} \quad \text{with } \text{RPO}^5(v_5 - v_6) = (v_1 - v_4 - v_3 - v_2 - v_5) \quad \text{and } \text{RPO}^5(v_5 - v_7) = (v_1 - v_4 - v_3 - v_6 - v_5),$$

$$R^6 = \{v_1 - v_4 - v_3 - v_6, v_1 - v_4 - v_3 - v_2 - v_5 - v_6\} \quad \text{with } \text{RPO}^6(v_6 - v_5) = (v_1 - v_4 - v_3 - v_6) \quad \text{and } \text{RPO}^6(v_6 - v_7) = (v_1 - v_4 - v_3 - v_2 - v_5 - v_6),$$

$$R^7 = \{v_1 - v_3 - v_4 - v_7\} \quad \text{with } \text{RPO}^7(\Delta r_k) = (v_1 - v_3 - v_4 - v_7),$$

where

$$[X^5(v_1 - v_4 - v_3 - v_6 - v_5), Y^5(v_1 - v_4 - v_3 - v_6 - v_5)] = [35, 121],$$

$$[X^6(v_1 - v_4 - v_3 - v_2 - v_5 - v_6), Y^6(v_1 - v_4 - v_3 - v_2 - v_5 - v_6)] = [21, 80].$$

- Since $|C| = 3 > 2$, Step 3 calls Algorithm 2 and uses the temporarily preferred (partial) paths in $R^\ell, v_\ell (\neq v_n) \in C = \{v_5, v_6, v_7\}$, to construct the preferred (partial) paths in $R^j, v_j (\neq v_\ell) \in C$, yielding

$$\begin{aligned}
R^5 &= \{v_1 - v_4 - v_3 - v_2 - v_5, v_1 - v_4 - v_3 - v_6 - v_5\} \quad \text{with } \text{RPO}^5(v_5 - v_6) \\
&= (v_1 - v_4 - v_3 - v_2 - v_5) \quad \text{and} \quad \text{RPO}^5(v_5 - v_7) = (v_1 - v_4 - v_3 - v_6 - v_5), \\
R^6 &= \{v_1 - v_4 - v_3 - v_6, v_1 - v_4 - v_3 - v_2 - v_5 - v_6\} \quad \text{with } \text{RPO}^6(v_6 - v_5) \\
&= (v_1 - v_4 - v_3 - v_6) \quad \text{and} \quad \text{RPO}^6(v_6 - v_7) = (v_1 - v_4 - v_3 - v_2 - v_5 - v_6), \\
R^7 &= \{v_1 - v_4 - v_3 - v_6 - v_5 - v_7\} \quad \text{with } \text{RPO}^7(\Delta r_k) = (v_1 - v_4 - v_3 - v_6 - v_5 - v_7),
\end{aligned}$$

where

$$[X^7(v_1 - v_4 - v_3 - v_6 - v_5 - v_7), Y^7(v_1 - v_4 - v_3 - v_6 - v_5 - v_7)] = [42, 129].$$

- Since $D = \emptyset$, Step 4 sets $B := B \cup C = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7\}$. Go to Step 5.
- Finally, Step 5 considers the only candidate left in $R^7 = \{v_1 - v_4 - v_3 - v_6 - v_5 - v_7\}$ as the optimal path r^* to the destination node v_7 . Hence, $r^* = v_1 - v_4 - v_3 - v_6 - v_5 - v_7$ with the minimum cost $f(X^7(r^*), Y^7(r^*)) = 0.325$ where $[X^7(r^*), Y^7(r^*)] = [42, 129]$. Terminate the algorithm.

Table 1 shows how the $\text{RPO}^\ell(\Delta r_k)$ and the TPI's among the temporarily preferred (partial) paths at each node $v_\ell \in V$ for various path segment additions Δr_k are utilized to find the optimal path between the O–D nodes v_1 and v_7 . This information is needed to construct (partial) paths to the successor nodes of each node. For instance, at node v_4 for the path segment addition $v_4 - v_6$ with $\Delta X_k = 8$, the $\text{RPO}^4(v_4 - v_6) = (v_1 - v_4, v_1 - v_3 - v_4)$ with $\text{TPI}^4(v_1 - v_4) = [0, 3.57)$ and $\text{TPI}^4(v_1 - v_3 - v_4) = [3.57, \infty)$, are used to construct the path $v_1 - v_3 - v_4 - v_6$ to node v_6 . Observe also that due to our preference structures and elimination processes a very significant number of paths are discarded leaving behind quite a small number of paths in the $\text{RPO}^\ell(\Delta r_k)$ at each node $v_\ell \in V$. As displayed in the third column of Table 1, for example, at node v_7 only one (out of forty five) path is kept in $\text{RPO}^7(\Delta r_k)$ at v_n as a candidate for the optimal path between the O – D nodes v_1 and v_7 .

As mentioned earlier, Algorithm 3 can be also extended to obtain the optimal paths from the origin node v_1 to all other nodes of a network. The results for the network of Fig. 6 are summarized in Table 2. The second column of the table shows that only one (out of five) path is kept in the $\text{RPO}^2(\Delta r_k)$ at node v_2 for further consideration. Similarly, the respective number of paths kept in the $\text{RPO}^\ell(\Delta r_k)$ at each other node is: one (of five) at node v_3 ; one (of five) at node v_4 ; one (of twenty) at node v_5 ; one (of twenty) at node v_6 ; and only one (of forty five) at node v_7 . That is, from a total of 100 paths from node v_1 to nodes $v_2, \dots, v_7$, ninety four of them are discarded by our elimination processes leaving behind only the six optimal paths to these nodes.

As discussed in Section 2, we may consider solving OPFCF by determining the entire set of Pareto-optimal paths between the O–D nodes v_1 and v_n of $\widehat{G}(V, A)$, and then choosing the Pareto path with the minimum cost as the optimal path. To generate these paths and thus to identify r^* , we develop a Pareto path algorithm (Algorithm 4). Basically, this algorithm applies the Pareto-optimality conditions for OPFCF (see Definition 2) to the (partial) paths to each node in order to find and keep the Pareto-optimal (partial) paths and to eliminate the non-Pareto (partial) paths to that node. The Pareto-optimal (partial) paths are then augmented by each arc connecting that node to each succeeding adjacent node in order to generate (partial) paths to that adjacent node. This process continues until all Pareto-optimal paths to the destination node are found and the optimal path is chosen. In developing Algorithm 4, we notice that its Steps 1, 2.b, 4, and 5 are the same as those of Algorithm 3 while its Steps 2.a and 3 are completely different. This is due to the use of the Pareto-optimality

Table 2

The RPO's among the temporarily preferred (partial) paths at each node when finding optimal paths from node v_1 to all other nodes of the bi-attribute network $\widehat{G}(V, A)$ of Fig. 6

Node	RPO	$[X_i, Y_i]$	Cost	Optimal path
v_2	$v_1 - v_4 - v_3 - v_2$	[13, 57]	0.228	$v_1 - v_4 - v_3 - v_2$
v_3	$v_1 - v_4 - v_3$	[11, 53]	0.207	$v_1 - v_4 - v_3$
v_4	$v_1 - v_4$	[9, 33]	0.272	$v_1 - v_4$
v_5	$v_1 - v_4 - v_3 - v_6 - v_5$	[35, 121]	0.289	$v_1 - v_4 - v_3 - v_6 - v_5$
v_6	$v_1 - v_4 - v_3 - v_2 - v_5 - v_6$	[21, 80]	0.262	$v_1 - v_4 - v_3 - v_2 - v_5 - v_6$
v_7	$v_1 - v_4 - v_3 - v_6 - v_5 - v_7$	[42, 129]	0.325	$v_1 - v_4 - v_3 - v_6 - v_5 - v_7$

conditions of [Definition 2](#), instead of [Algorithm 2](#), to keep the Pareto-optimal (partial) paths and to discard the non-Pareto (partial) paths at each node. A formal presentation of [Algorithm 4](#) is given in [Appendix](#).

Applying [Algorithm 4](#) to the network of [Fig. 6](#), we find the Pareto path $v_1 - v_4 - v_3 - v_6 - v_5 - v_7$ with cost 0.325 to be optimal (i.e., $r^*: v_1 - v_4 - v_3 - v_6 - v_5 - v_7$). In the process of finding r^* , we observe that three (out of five) paths to v_2 , two (out of five) paths to v_3 , three (out of five) paths to v_4 are Pareto-optimal and the remaining two, three, and two paths to v_2 , v_3 , and v_4 , respectively, are non-Pareto and thus discarded due to Steps 2.a and 3 of the algorithm. Of the 20 paths to v_5 , only five are Pareto-optimal and the non-Pareto paths are eliminated. Also, of the 20 paths to v_6 , five are Pareto-optimal. Finally, of the 45 paths to v_7 , nine are Pareto-optimal. Evaluating the costs of these nine paths, the algorithm finds $r^*: v_1 - v_4 - v_3 - v_6 - v_5 - v_7$.

6. Computational results

In this section, we present some computational experiments to demonstrate and compare the performance of the proposed optimal path algorithm ([Algorithm 3](#)) with that of the Pareto path algorithm ([Algorithm 4](#)) for solving OPFCF.

Our computational results on determining optimal paths for OPFCF are carried out on two types of networks $\hat{G}(V, A)$: random networks and grid networks. These network topologies are well organized and have been used extensively to test algorithms for the single criterion and multiple criteria shortest path problems (e.g. [\[18,25,39,41,43\]](#)). For a random network, a Hamiltonian cycle is first randomly generated to guarantee connectivity of the network, and then the remaining $|A| - |V| = m - n$ arcs are randomly generated with multiple arcs being excluded. For a grid network, the nodes correspond to the points of a square or rectangular mesh in which each pair of adjacent nodes is connected by an arc. Additional arcs are generated with random starting and ending nodes. The origin node v_1 , being node 1 , is placed at the top-left corner while the destination node v_n , being node $n = |V|$, is located at the bottom right corner of the planar grid.

In our experiments, we have randomly generated three different sets of arc attributes x and y for each network $\hat{G}(V, A)$. In the first set, both x and y attributes for every arc are sampled from a uniform distribution in the range from $[1, 100]$ yielding uncorrelated x and y (i.e., $\rho_{x,y} \approx 0$). In the second set, the x and y attributes for an arc are respectively sampled from uniform distributions in the ranges from $[1, 100]$ and $[200, 300]$ and those

Table 3
Characteristics of the random networks $\hat{G}(V, A)$

Problem	$ V $: no. of nodes	$ A $: no. of arcs
R11	196	1500
R12	196	3000
R13	196	4500
R14	196	6000
R21	256	1500
R22	256	3000
R23	256	4500
R24	256	6000
R31	400	1500
R32	400	3000
R33	400	4500
R34	400	6000
R41	576	1500
R42	576	3000
R43	576	4500
R44	576	6000
R51	676	1500
R52	676	3000
R53	676	4500
R54	676	6000

for another arc are respectively sampled from uniform distributions in the ranges from [200, 300] and [1, 100]; thus, negatively correlated x and y (i.e., $\rho_{x,y} < 0$) are generated. The third set contains positively correlated x and y (i.e., $\rho_{x,y} > 0$) where both attributes for an arc are sampled from a uniform distribution in the range from [1, 100] and those for another arc from a uniform distribution in the range from [200, 300]. Negatively (positively) correlated attributes imply less (more) temporarily preferred paths for the optimal path algorithm (Algorithm 3) as well as less (more) Pareto-optimal or efficient paths for the Pareto path algorithm (Algorithm 4) (see Sections 2,3 and 5).

We have generated five sets of undirected random networks by using the Netmaker of Skriver and Andersen [43]. Each set contains four networks having the same number of nodes but different number of arcs. The characteristics of the 20 random networks (R11–R54) are given in Table 3. In addition, we have used the Gridgen generator of Bertsekas [6] to generate 36 undirected grid networks whose characteristics are reported in Table 4. These include 12 square grid networks (G111–G113, G211–G213, G311–G313, and G411–G413) and 24 rectangular grid networks (G121–G133, G221–G233, G321–G333, and G421–G433).

Table 5 shows the computational results on the 20 OPFCF problems R11–R54 of Table 3 where 10 networks for every problem have been randomly generated. Each problem was tested using the three generated sets of uncorrelated, negatively correlated, and positively correlated arc attributes. We solved the resulting six

Table 4

Characteristics of the square and rectangular grid networks $\hat{G}(V, A)$

Problem	$ V $: no. of nodes	Dimension	$ A $: no. of arcs
G111	196	14×14	1500
G112	196	14×14	3000
G113	196	14×14	4500
G121	196	7×28	1500
G122	196	7×28	3000
G123	196	7×28	4500
G131	196	4×49	1500
G132	196	4×49	3000
G133	196	4×49	4500
G211	256	16×16	1500
G212	256	16×16	3000
G213	256	16×16	4500
G221	256	8×32	1500
G222	256	8×32	3000
G223	256	8×32	4500
G231	256	4×64	1500
G232	256	4×64	3000
G233	256	4×64	4500
G311	400	20×20	1500
G312	400	20×20	3000
G313	400	20×20	4500
G321	400	10×40	1500
G322	400	10×40	3000
G323	400	10×40	4500
G331	400	5×80	1500
G332	400	5×80	3000
G333	400	5×80	4500
G411	576	24×24	1500
G412	576	24×24	3000
G413	576	24×24	4500
G421	576	12×48	1500
G422	576	12×48	3000
G423	576	12×48	4500
G431	576	6×96	1500
G432	576	6×96	3000
G433	576	6×96	4500

hundred test problems by the proposed optimal path algorithm (Algorithm 3) and the Pareto path algorithm (Algorithm 4). The programs were written in Fortran 77 and run on an HP/Alpha GS 80 computer with VMS operating system. The CPU times (in seconds) are the averages of 10 runs. Given the NP completeness of the problem, our computational results undoubtedly demonstrate that the proposed optimal path algorithm solves different size problems in a very reasonable amount of time. As expected, these times are clearly much smaller than those of the alternative algorithm (i.e., Algorithm 4) which generates and evaluates all Pareto-optimal paths. The results also show that the type of correlation among arc attributes affects the CPU times. The CPU times for the case where $\rho_{x,y} < 0$ ($\rho_{x,y} > 0$) are smaller (larger) than those for the case where $\rho_{x,y} \approx 0$ due to the fact that fewer (more) temporarily preferred paths in Algorithm 3 and less (more) Pareto-optimal paths in Algorithm 4 are generated, evaluated, and stored. Moreover, as the numbers of nodes and arcs in a network increase, the increase in the CPU time of Algorithm 3 becomes very insignificant compared to that of Algorithm 4. For example, for problem R22 (see Table 3) the CPU times of Algorithm 3 averaged 13.62, 19.48, and 27.45 seconds for $\rho_{x,y} < 0$, $\rho_{x,y} \approx 0$, and $\rho_{x,y} > 0$, respectively; while the corresponding times for Algorithm 4 were 199.41, 287.25, and 398.33. For problem R54 (defined on a larger network) the average CPU times of Algorithm 3 were, respectively, 78.96, 110.67, and 154.39 seconds, while those of Algorithm 4 were 1,796.55, 3,734.48, and 5,141.13 seconds.

Table 6 displays our computational results on the 36 OPFCF problems G111–G433 of Table 4 where 10 networks for every problem have been generated. Each problem was also tested using the three sets of uncorrelated, negatively correlated, and positively correlated arc attributes. The resulting one thousand eighty test problems were solved by Algorithms 3 and 4. As the results of Table 6 show, a trend similar to that for the random networks is also observed for the grid networks. For example, for problem G111 defined on a square network (see Table 4), the average CPU times of Algorithm 3 were 7.21, 10.78, and 13.75 seconds for $\rho_{x,y} < 0$, $\rho_{x,y} \approx 0$, and $\rho_{x,y} > 0$, respectively; while the corresponding times of Algorithm 4 were 84.88, 127.37, and 187.64 seconds. For problem G413 (defined on a larger square network) these times increased to 45.31, 62.46, and 87.18 for Algorithm 3, and to 680.16, 984.52, 1925.40 seconds for Algorithm 4. In addition, for problem G133 defined on a rectangular network, the CPU times of Algorithm 3 averaged 51.96, 74.88, and

Table 5

Computational results on the optimal path problem with fractional cost functions on the random networks of Table 3 using negatively correlated, uncorrelated, and positively correlated arc attributes

Problem	Optimal path algorithm (Algorithm 3)			Pareto path algorithm (Algorithm 4)		
	$\rho_{x,y} < 0$	$\rho_{x,y} \approx 0$	$\rho_{x,y} > 0$	$\rho_{x,y} < 0$	$\rho_{x,y} \approx 0$	$\rho_{x,y} > 0$
R11	6.14	8.57	14.21	67.95	101.46	149.37
R12	12.59	18.68	25.14	182.69	263.50	367.92
R13	24.83	35.95	49.02	529.17	785.36	1095.21
R14	57.15	79.09	107.73	1847.65	2,775.18	3923.04
R21	6.40	9.11	12.94	74.26	109.39	163.06
R22	13.62	19.48	27.45	199.41	287.25	398.33
R23	28.19	41.15	56.52	593.24	842.36	1202.42
R24	60.43	83.74	112.58	2087.38	3,045.41	4158.65
R31	7.15	10.35	14.65	84.36	122.46	187.40
R32	15.24	22.50	32.92	219.53	306.11	431.81
R33	33.51	45.89	62.08	658.54	912.64	1296.37
R34	67.33	91.51	125.46	2313.86	3258.35	4471.28
R41	7.92	11.44	16.68	96.73	141.22	208.45
R42	17.57	26.88	35.15	234.25	331.45	469.32
R43	37.18	51.42	72.57	724.48	982.39	1364.73
R44	72.29	101.15	138.94	2585.26	3424.65	4750.56
R51	9.44	13.25	20.32	108.53	157.24	222.61
R52	20.01	28.64	40.86	255.06	359.77	511.40
R53	40.27	55.86	79.23	782.32	1056.11	1529.75
R54	78.96	110.67	154.39	1796.55	3734.48	5141.13

The CPU times (in seconds) are averages of 10 runs.

Table 6

Computational results on the optimal path problem with fractional cost functions on the square and rectangular grid networks of Table 4 using negatively correlated, uncorrelated, and positively correlated arc attributes

Problem	Optimal path algorithm (Algorithm 3)			Pareto path algorithm (Algorithm 4)		
	$\rho_{x,y} < 0$	$\rho_{x,y} \approx 0$	$\rho_{x,y} > 0$	$\rho_{x,y} < 0$	$\rho_{x,y} \approx 0$	$\rho_{x,y} > 0$
G111	7.21	10.78	13.75	84.88	127.37	187.64
G112	14.01	19.35	27.72	204.16	295.92	414.05
G113	27.14	37.62	54.87	621.43	872.17	1,196.73
G121	9.14	13.43	17.61	107.82	161.95	238.88
G122	17.66	26.14	35.32	258.05	377.69	528.34
G123	34.29	49.25	69.57	787.34	1105.23	1547.98
G131	13.24	18.31	25.82	165.73	250.34	363.69
G132	26.45	37.52	54.65	401.15	579.28	814.61
G133	51.96	74.88	108.73	1221.91	2049.75	2816.46
G211	8.51	11.24	17.13	98.06	145.53	217.62
G212	15.79	22.48	32.26	230.84	332.36	464.85
G213	31.23	45.44	62.19	698.43	992.47	1381.52
G221	10.51	14.89	20.63	123.78	183.22	272.28
G222	19.28	27.15	40.05	290.06	418.56	584.98
G223	39.82	57.66	79.36	882.53	1,259.42	1,739.34
G231	17.81	24.17	34.28	207.74	309.56	460.64
G232	32.65	46.21	68.17	488.76	708.28	992.49
G233	65.03	95.55	134.82	1475.26	2196.85	2983.57
G311	10.25	15.93	22.49	126.54	181.76	282.59
G312	18.13	27.48	42.61	273.15	384.05	542.74
G313	39.46	54.61	73.12	849.96	1125.42	1625.15
G321	12.93	19.12	29.87	158.79	231.96	348.16
G322	22.68	33.46	54.25	342.93	482.85	683.31
G323	49.14	68.05	95.41	1053.57	1458.64	2091.75
G331	21.75	33.16	49.35	273.68	401.56	586.88
G332	39.24	58.75	90.55	594.13	838.28	1,176.42
G333	85.02	118.33	163.18	1771.32	2438.51	3539.16
G411	13.76	19.18	28.87	165.34	244.19	365.29
G412	23.81	35.35	52.45	319.58	453.38	644.12
G413	45.31	62.46	87.18	680.16	984.52	1925.40
G421	17.61	23.74	34.47	206.72	306.35	457.45
G422	30.22	44.03	66.04	401.94	572.74	810.51
G423	55.17	78.48	112.42	892.19	1212.12	2343.56
G431	29.71	41.67	61.58	357.66	526.81	690.69
G432	52.08	75.52	113.23	693.47	985.95	1345.23
G433	97.65	135.16	190.49	1567.32	2114.56	3857.58

The CPU times (in seconds) are averages of 10 runs.

108.73 seconds for $\rho_{x,y} < 0$, $\rho_{x,y} \approx 0$, and $\rho_{x,y} > 0$, respectively; while the times of Algorithm 4 were 1,221.91, 2,049.75, and 2,816.46 seconds. For problem G433 (defined on a larger rectangular network) these times were 97.65, 135.16, and 190.49 seconds for Algorithm 3, and 1567.32, 2114.56, 3857.58 seconds for Algorithm 4. Indeed, the CPU times of the proposed optimal path algorithm (Algorithm 3) for this NP-complete problem are very appealing.

7. Summary and conclusions

In this paper, we have defined and formulated an optimal path problem in a bi-attribute network with fractional cost function (OPFCF). The objective is to identify an optimal simple path (i.e., an optimal path with no cycles) between an origin and a destination that minimizes the ratio of a linear function of one criterion and a linear function of the other criterion. OPFCF is an NP-complete problem since it can be equivalently formulated as the “bi-attribute rational path problem”. Due to the differences in the Pareto-optimality conditions

for OPFCF and those for the multicriteria shortest path problem (MSPP), we have demonstrated that neither the standard methods for MSPP nor a DP-based procedure can be applied to solve OPFCF. However, in this paper, we have developed an exact solution approach for OPFCF which finds an optimal path when arc attributes are non-negative. The proposed approach, which is based on some path preference concepts including permanent and temporary preferences, relative preference orderings, and temporary preference intervals among paths, effectively utilizes a number of elimination techniques to discard, from further consideration, those (partial) paths that cannot be parts of an optimal path. These elimination techniques have been used to introduce two algorithms (Algorithms 1 and 2) in order to find optimal paths in some special networks. Algorithm 1 identifies and eliminates (partial) paths at every node and then uses the relative preference ordering and temporary preference intervals among the remaining (partial) paths at the node in order to construct (partial) paths to the succeeding adjacent nodes of that node. Algorithm 2 finds optimal paths for the special networks by calling Algorithm 1 at every node including the destination. A label correcting type algorithm (Algorithm 3) has been proposed to find optimal paths in general networks. This algorithm utilizes the path preference information obtained from Algorithm 2 in order to identify an optimal path. In its algorithmic process, Algorithm 3 compares the preference information of (partial) paths at each node in order to identify and eliminate those (partial) paths that cannot be parts of candidates for the optimal path at the destination. This process significantly reduces the number of (partial) paths generated, evaluated, and stored in the search for finding an optimal path. We have also presented a Pareto path algorithm (Algorithm 4) which can be used to generate all Pareto-optimal paths in a general network in order to choose an optimal Pareto path. Our extensive computational results on Algorithms 3 and 4 experimenting with random and grid (both square and rectangular) networks with 196–676 nodes and 1500–6000 as well as uncorrelated, negatively correlated, and positively correlated arc attributes have demonstrated that (i) Algorithm 3 finds optimal paths in a very reasonable amount of time indicating the importance of the proposed elimination techniques, (ii) the CPU times of Algorithm 3 are significantly lower than those of Algorithm 4, (iii) the CPU times for networks with negatively (positively) correlated arc attributes are smaller (larger) than those with uncorrelated arc attributes due to the fact that less (more) temporarily preferred (partial) paths in Algorithm 3 and less (more) Pareto-optimal paths in Algorithm 4 are generated, evaluated, and stored, and (iv) as the numbers of nodes and arcs in a network increase, the increase in the CPU time of Algorithm 3 becomes very insignificant compared to that of Algorithm 4. Given the NP-completeness of OPFCF, the computational results reveal the effectiveness of Algorithm 3 for solving quite large problems. These results also show the deficiency of Algorithm 4 since generating all Pareto-optimal paths in OPFCF may be computationally intractable. Finally, this research is being currently extended to develop an algorithm for the optimal path problem with fractional cost function in a bi-attribute network whose arc attributes is not necessarily non-negative. Further research is also needed to present an optimal path algorithm for multicriteria networks with arcs having more than two attributes. In the absence of such an algorithm, the development of some efficient heuristics is highly recommended.

Acknowledgements

The author expresses his gratitude to the three anonymous referees for their valuable comments. Also, special thanks to the Office of Vice President for Research at Kuwait University for funding this research under grant SS039.

Appendix

Algorithm 4 (*A Pareto path algorithm*).

Begin

1. $B := \{v_1\};$
 $C := \{\text{adjacent nodes to } v_1 \in B\};$
 $D := V - (B \cup C);$
 $R^1 = \emptyset;$
 $[X^1(r), Y^1(r)] = [0, 0] \forall r \in R^1;$

2. a. **For** ($v_\ell \in B$, and $v_\ell \neq v_1$, and $v_j \in C$, and R^ℓ) **do**
 Compute $[X^j(r_k^j), Y^j(r_k^j)], r_k^j \in R^j$ using Eq. (2);
 End do
 For ($r_{k_1}^j \in R^j$, and $r_{k_2}^j \in R^j$) **do**
 If ($X^j(r_{k_1}^j) < X^j(r_{k_2}^j)$ and $Y^j(r_{k_1}^j) \geq Y^j(r_{k_2}^j)$) or
 ($X^j(r_{k_1}^j) \leq X^j(r_{k_2}^j)$ and $Y^j(r_{k_1}^j) > Y^j(r_{k_2}^j)$) **then**
 $R^j := R^j - \{r_{k_2}^j\}$;
 End if
 End do
b. **Repeat**
 Step 2.a for each $v_\ell (\neq v_n) \in C$ and each $v_j (\neq v_\ell) \in C$;
 Until (no more new (partial) paths can be included in $R^j, v_j \in C$);

3. **If** $|C| \leq 2$ **then**
 goto Step 4
 Else
 For ($v_\ell \in C$, and $v_j \in C$, and R^ℓ) **do**
 Compute $[X^j(r_k^j), Y^j(r_k^j)], r_k^j \in R^j$ using Eq. (2);
 End do
 End if

4. Close each $v_\ell \in C$ if v_ℓ and none of its predecessors $v_j \in C$ have any incoming arcs from a $v_k \in D$;
 If $D = \emptyset$ **then**
 $B := B \cup C$;
 goto Step 5
 Else

a. **If** (all nodes $v_\ell \in C$ can be closed) **then**
 $B := B \cup C, C := \{all\ v_k \in D\ adjacent\ to\ each\ v_j \in B\}$;

b. **Else** (no node $v_\ell \in C$ can be closed)
 $C := C \cup \{all\ v_k \in D\ adjacent\ to\ each\ v_j \in C\}$;
 End if
 $D := V - (B \cup C)$;
 goto Step 2
 End if

5. $r^* := \underset{r_k^n \in R^n}{\operatorname{argmin}} \{f(X^n(r_k^n), Y^n(r_k^n))\} = \underset{r_k^n \in R^n}{\operatorname{argmin}} \{X^n(r_k^n)/Y^n(r_k^n)\}$;

Exit

References

- [1] R.K. Ahuja, Minimum cost-reliability ratio path problem, *Computers and Operations Research* 15 (1988) 83–89.
- [2] R.K. Ahuja, J.L. Batra, S.K. Gupta, Combinatorial optimization with rational objective functions: a communication, *Mathematics of Operations Research* 8 (1983) 314.
- [3] C.H. Antunes, J. Craveirinha, J. Climaco, C. Barrico, Multiple objective routing in integrated communication networks, in: D. Smith, P. Key (Eds.), *Teletraffic engineering in a competitive world*, Proceedings of the 16th International Teletraffic Congress—ITC 16, Teletraffic Science and Engineering, vol. 3b, Elsevier, 1999, pp. 1291–1300.
- [4] J.A. Azevedo, E.Q.V. Martins, An algorithm for the multiobjective shortest path problem on acyclic networks, *Investigacao Operational* 11 (1991) 52–69.
- [5] R. Batta, S.S. Chiu, Optimal obnoxious paths on a network: Transportation of hazardous materials, *Operations Research* 36 (1988) 84–92.
- [6] D.P. Bertsekas, A simple and fast label correcting algorithm for shortest paths, *Network* 23 (1993) 703–709.
- [7] R. Carraway, L. Morin, H. Moskowitz, Generalized dynamic programming for multicriteria optimization, *European Journal of Operational Research* 44 (1990) 95–104.
- [8] J.C.N. Climaco, E.Q.V. Martins, On the determination of the nondominated paths in a multiobjective network problem, *Methods in Operations Research* 40 (1981) 255–258.
- [9] J.C.N. Climaco, E.Q.V. Martins, A bicriterion shortest path algorithm, *European Journal of Operational Research* 11 (1982) 399–404.

- [10] J.C.N. Climaco, J.M.F. Craveirinha, M.M.B. Pascoal, A bicriterion approach for routing problems in multimedia networks, *Networks* 41 (2003) 206–220.
- [11] J.M. Coutinho-Rodrigues, J.C.N. Climaco, J.R. Current, A PC-based interactive decision support system for two objective direct delivery problems, *Journal of Business Logistics* 15 (1994) 305–322.
- [12] J.M. Coutinho-Rodrigues, J.C.N. Clímaco, J.R. Current, An interactive bi-objective shortest path approach: Searching for unsupported nondominated solutions, *Computers and Operations Research* 26 (1999) 789–798.
- [13] R.G. Cox, *Routing and Scheduling of Hazardous Materials Shipments: Algorithmic Approaches to Managing Spent Nuclear Fuel Transportation*, Ph.D. Dissertation, Cornell University, Ithaca, NY, 1984.
- [14] J.R. Current, M. Marsh, Multiobjective transportation network design and routing problems: Taxonomy and annotation, *European Journal of Operational Research* 103 (1993) 426–438.
- [15] J.R. Current, H. Min, Multiobjective design of transportation networks: Taxonomy and annotation, *European Journal of Operational Research* 26 (1986) 187–201.
- [16] J.R. Current, C.S. ReVelle, J.L. Cohon, The median shortest path problem: A multiobjective approach to analyze cost vs. accessibility in the design of transportation networks, *Transportation Science* 21 (1987) 490–503.
- [17] J.R. Current, C.S. ReVelle, J.L. Cohon, An interactive approach to identify the best compromise solution for two objective shortest path problems, *Computers and Operations Research* 17 (1990) 187–198.
- [18] N. Deo, C.Y. Pang, Shortest path problems: Taxonomy and annotation, *Networks* 14 (1984) 275–323.
- [19] E.W. Dijkstra, A note on two problems in connection with graphs, *Numerische Mathematik* 1 (1959) 269–271.
- [20] S.E. Elmaghraby, *Activity Networks: Project Planning and Control by Network Models*, Wiley, New York, 1977.
- [21] J.R. Evans, E. Minieka, *Optimization Algorithms for Networks and Graphs*, second ed., Marcel Dekker, Inc., New York, 1992.
- [22] B.L. Fox, Finding minimum time–cost ratio circuits, *Operations Research* 17 (1969) 546–551.
- [23] K. Fujimura, Path planning with multiple objectives, *IEEE Robotics and Automation Magazine* (1996) 33–38.
- [24] M. Garey, D. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, Freeman, San Francisco, CA, 1979.
- [25] J. Granat, F. Guerriero, The interactive analysis of the multicriteria shortest path problem by the reference point method, *European Journal of Operational Research* 151 (2003) 103–118.
- [26] C. Hallam, K.J. Harrison, J.A. Ward, A multiobjective optimal path algorithm, *Digital Signal Processing* 11 (2001) 133–143.
- [27] P. Hansen, Bicriterion path problems, in: G. Fandel, T. Gal (Eds.), *Multiple Criteria Decision Making Theory and Applications: Lecture Notes in Economics and Mathematical Systems*, vol. 177, Springer-Verlag, Berlin, 1979.
- [28] M.I. Henig, The shortest path problem with two objective functions, *European Journal of Operational Research* 25 (1985) 281–291.
- [29] H. Katzan, *Distributed Information Systems*, Petrocelli, New York, 1979.
- [30] E.Q.V. Martins, On a special class of bicriterion path problems, *European Journal of Operational Research* 17 (1984) 85–94.
- [31] E.Q.V. Martins, On a multicriteria shortest path problem, *European Journal of Operational Research* 16 (1984) 236–245.
- [32] E.Q.V. Martins, An algorithm to determine a path with minimal cost/capacity ratio, *Discrete Applied Mathematics* 8 (1984) 189–194.
- [33] E.Q.V. Martins, J.L.E. Santos, The Labeling Algorithm for the Multiobjective Shortest Path Problem, Departamento de Matematica, Universidade de Coimbra, Portugal, 1999.
- [34] E.Q.V. Martins, J.L.E. Santos, An algorithm for the quickest path problem, *Operations Research Letters* 20 (1997) 195–198.
- [35] N. Meggido, Combinatorial optimization with rational objective functions, *Mathematics of Operations Research* 4 (1979) 414–424.
- [36] P.B. Mirchandani, H. Soroush, Optimal paths in probabilistic networks: A case with temporary preferences, *Computers and Operations Research* 12 (1985) 365–381.
- [37] P.B. Mirchandani, M. Wiecek, Routing with nonlinear multi-attribute cost functions, *Applied Mathematics and Computers* 54 (1993) 215–239.
- [38] P. Modesti, A. Sciomachen, A utility measure for finding multiobjective shortest paths in urban multimodal transportation networks, *Proceedings of AIRO* (1996) 615–617.
- [39] J. Mote, I. Murthy, D.L. Olson, A parametric approach to solving bicriterion shortest path problems, *European Journal of Operational Research* 53 (1991) 81–92.
- [40] I. Murthy, D.L. Olson, An interactive procedure using domination cones for bicriterion shortest path problems, *European Journal of Operational Research* 72 (1999) 417–431.
- [41] J.M.P. Paixao, E.Q.V. Martins, M.S. Rosa, J.L.E. Santos, The determination of the path with minimum-cost norm value, *Networks* 41 (2003) 184–196.
- [42] H.D. Sherali, L.D. Brizendine, T.S. Glickman, S. Subramanian, Low probability-high consequence considerations in routing hazardous material shipments, *Transportation Science* 31 (1997) 237–251.
- [43] A.J.V. Skriver, K.A. Anderson, A label correcting approach for solving bicriterion short-path problems, *Computers and Operations Research* 27 (2000) 507–524.
- [44] C.T. Tung, K.L. Chew, A multicriteria Pareto-optimal path problem, *European Journal of Operational Research* 62 (1992) 203–209.
- [45] A. Warburton, Approximation of Pareto optima in multiple objective shortest path problems, *Operations Research* 35 (1987) 70–79.
- [46] D.J. White, *Dynamic Programming*, Holden-Day, San Francisco, 1969.